作品编号：CACR20254UR1DK

作品类别：☐软件设计　☐硬件制作　☑工程实践

☐密码应用技术　☐其它

2025 年第十届全国密码技术竞赛作品设计报告

作品题目：车联网级后量子密码算法融合优化方案

2025 年 10 月 21 日

中国密码学会

基本信息表
编号：CACR20254UR1DK
作品题目：车联网级后量子密码算法融合优化方案
作品类别：☐软件设计　☐硬件制作　☑工程实践　☐密码技术应用　☐其它
作品内容摘要： 智能网联汽车的迅速发展扩大了通信攻击面，国家推动“密码迁移”，加速后量子密码（PQC）标准落地。然而，PQC算法计算复杂度高、资源开销大，难以适配嵌入式设备。现有硬件优化方案多聚焦于单一算法，导致多算法共存时资源冗余严重、系统扩展性不足。 为应对上述挑战，本作品提出一种可兼容Kyber与Dilithium算法的车联网级硬件融合优化方案。在哈希、采样、多项式三个关键环节中，构建以总面积最小化为约束、计算效率最大化为目标的规划求解模型，实现系统整体计算效率的全局最优。具体改进如下： （1）重构哈希加速器：为优化传统计算的吞吐受限问题，在核心计算模块中搭建双阶段流水线的层次调度架构，支持常规路径与冻结－恢复双模式的智能状态机；在子步骤级基于硬件特性，采用逆向坐标重映射实现步骤融合的无逻辑级路径转移，从而降低逻辑深度。在等效资源配置下，该模块相比传统单周期设计，吞吐率提升约3.1倍，节省约21%的查找表资源和42%的触发器，显著提升计算密度。 （2）轻量级采样融合器：为解决存储资源开销大的问题，针对三类采样方式制定四级模块化标准，通过三级调度寄存器复用，使控制信号量减少66%以上，并节省至少17个寄存器，显著降低硬件资源占用与系统功耗。 （3）多项式存算一体架构：设计D-KPolyArch架构，解决两种算法在结构差异与资源复用方面的挑战。在存储侧，采用交织式4-Bank结构与混合进制地址映射，实现零冲突并行访存；在计算侧，构建双 ALU 解耦架构，优化不同算法的多项式乘加运算路径，并结合统一流水线提升执行效率。与双核独立实现方案相比，该架构在等效资源下 吞吐率提升1.9倍，乘法器复用率提高32%，显著增强系统性能密度与融合部署能力。 此外，引入基于物理不可克隆函数（PUF）的多级种子管理机制，以DRAM-PUF输出可信根种子，通过分层派生实现随机数的按需动态生成与更新，有效降低密钥泄露风险。 在 ZYNQ 嵌入式平台的综合评测表明，相比主流软件库实现，本作品在执行Kyber与Dilithium算法的吞吐率和时延等性能上均提升约3倍；与高效GPU实现相比，系统平均达到其70%的性能，兼具高吞吐与低面积优势。性能指标满足车规级芯片的实时性与可靠性要求，为后量子密码算法在智能网联汽车与边缘设备中的安全部署提供了可行的硬件实现路径。
关键词：**后量子密码 车联网安全 统一算子架构 硬件加速 面积资源优化**

目录

第一章 作品概述 1
1.1 研究背景 1
1.2 作品特色 3

第二章 背景知识 5
2.1 密码算法介绍 5
2.1.1 格密码 5
2.1.2 Keccak 函数 6
2.2 多项式商环乘法的优化理论 7
2.2.1 模约简和模乘法的优化 7
2.2.2 基于 NTT 的多项式乘法优化 8

第三章 核心技术实现 11
3.1 多级种子管理机制 11
3.2 基于硬件的重构哈希加速器 12
3.2.1 双阶段流水的层次调度架构 13
3.2.2 子步骤级融合重构 FTS 16
3.3 寄存器复用的采样模块划分 20
3.4 D-KPolyArch 存储架构 27
3.4.1 D-K 存储架构设计 28
3.4.2 双 ALU 解耦设计 33

第四章 作品测试与分析 38
4.1 基于规划求解的算子结构最优构型 38
4.2 测试环境 39
4.3 关键模块测试 40
4.3.1 keccak 模块测试 40
4.3.2 多项式测试 41
4.4 系统正确性测试 43
4.5 总体性能测试 44
4.5.1 核心模块性能测试 44
4.5.2 系统性能测试 45

第五章 总结与展望 47

参考文献 48

图 目录

图 1.1 车联网示意图 1
图 2.1 后量子密码算法的性能对比 5
图 2.2 密钥生成算法的共享算子 6
图 2.3 Plantard 模约简逻辑电路 7
图 2.4 Barret 模约简逻辑电路 8
图 2.5 REDC 逻辑电路 8
图 2.6 CT 蝶形单元电路（左) 和 GS 蝶形单元电路（右) 9
图 2.7 8 系数 CT 蝶形网络 9
图 2.8 8 系数 GS 蝶形网络 10
图 3.1 DRAM PUF 的工作原理 11
图 3.2 子采样种子的生成原理 12
图 3.3 Keccak 设计总架构 13
图 3.4 DSRL 架构流水线实现流程图 14
图 3.5 并发任务状态调度结构 14
图 3.6 统一主状态机 15
图 3.7 多消息调度器的核心工作流程 15
图 3.8 两步融合优化架构 17
图 3.9 多级逻辑硬件实现方式对比 18
图 3.10 稀疏异或执行流程图 20
图 3.11 均匀随机采样硬件模块设计 22
图 3.12 均匀随机采样 Module3 执行逻辑 22
图 3.13 均匀随机采样 Module3 调度逻辑 23
图 3.14 中心二项分布采样 Module3 调度逻辑 24
图 3.15 SampleInBall 采样 Module3 调度逻辑 25
图 3.16 协同采样硬件模块设计 26
图 3.17 统一采样系统与独立采样的数据对比 27
图 3.18 D-KPolyArch 架构设计 28
图 3.19 D-K 存储架构设计 29
图 3.20 多项式矩阵-向量乘加过程的数据流路径 32
图 3.21 Montgomery 域转换路径 34

图 3.22 NTT 操作路径 34
图 3.23 INTT 操作路径 34
图 3.24 CWM 操作路径 34
图 3.25 Kyber 的 CWM 流水线设计 35
图 3.26 Kyber 的 ALU 设计 35
图 3.27 MAD 操作路径 36
图 3.28 Dilithium 的 ALU 设计 36
图 3.29 统一 5 级流水线设计 37
图 4.1 SHAKE-256 硬件模块级仿真结果 41
图 4.2 SHAKE-256 软件实现输出结果 41
图 4.3 Kyber 矩阵多项式运算仿真 1 42
图 4.4 Kyber 矩阵多项式运算仿真 2 42
图 4.5 Kyber 矩阵多项式运算仿真 3 42
图 4.6 Kyber 密钥生成结果 43
图 4.7 Kyber 密钥封装结果 43
图 4.8 Kyber 密钥解封装结果 43
图 4.9 Dilithium 签名结果示例 44
图 4.10 系统性能对比 46

表 目录

表 1.1 各企业在车联网信息安全上的进展 1
表 3.1 Kyber 与 Dilithium 中的子采样种子 12
表 3.2 简化后的轮常数表 19
表 3.3 Kyber 与 Dilithium 中的采样 21
表 3.4 采样机制的四级处理模块 21
表 3.5 3-RAM 结构 32
表 4.1 规划求解模型中的变量与参数定义 38
表 4.2 模型求解使用的算子参数 39
表 4.3 软硬件平台配置环境 40
表 4.4 各 Keccak 实现方案资源与性能对比 45
表 4.5 各多项式实现方案资源与性能对比 45
表 4.6 不同密码库每秒操作次数对比（单位：ops/s） 45
表 4.7 与 CCM3310S-T 车载芯片性能对比 46

第一章 作品概述

1.1 研究背景

(1) 车联网密码硬件的现状与发展趋势

智能网联汽车的快速发展使整车电子架构趋于复杂化，显著扩大了车辆信息系统的潜在攻击面[1]。未来，车联网安全体系重新面临数据传输的机密性、完整性与真实性要求。在将于 2026 年 1 月 1 日正式实施的国家强制性标准《汽车整车信息安全技术要求》中明确规定车辆应当“采用符合国际、国家或行业标准要求的密码模块”以实现加密通信、身份认证以及证书有效性验证等功能。

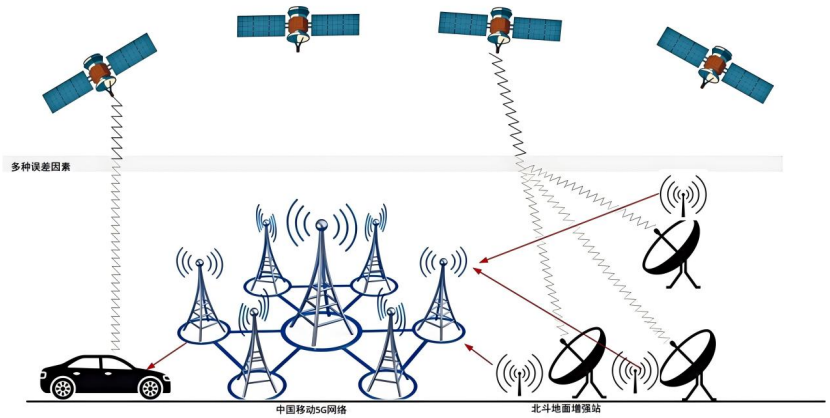


图 1.1　车联网示意图

表 1.1　各企业在车联网信息安全上的进展

企业名称	密码算法	优化策略	优化成果
瑞萨电子	ECC	RH850 单片机集成 HSM	减少软件代码，CPU 加速
恩智浦	ECIES	提供专用 SXF1800 元件	加速验签，节省 SoC 面积
英飞凌	ECDSA	AURIX™ 单片机集成 HSM	加速签名，节省 PCB 面积
高通	ECDSA	SA415M 芯片集成 HSM	加速验签，优化 BOM 成本
华大电子	SM4	专用芯片 CIU98$_{B/H}$	主控无需集成国密逻辑

如表 1.1所示，为满足上述需求，业界正积极探索硬件安全模块（HSM）和可信平台模块（TPM）等密码硬件的集成化部署与优化，其目标是在保障密钥管理、数据加密和身份认证等核心功能的前提下，提升算法运行效率，降低面积开销。此外，为进一步提升资源利用率，针对多个密码算法共享硬件算子的方案已有较为成熟的研究与工业应用[2]，例如统一椭圆曲线硬件加速器能够通过共享底层的有限域算术逻辑单

元 (ALU)，实现 ECIES 加密与 ECDSA 数字签名的资源复用，减少芯片面积。

(2) 后量子密码的挑战与研究动态

然而，现有的硬件优化方案主要集中于传统公钥密码算法[3][4]。随着量子计算机的发展，传统公钥密码算法的安全性将逐步削弱且其优化策略难以直接迁移至后量子密码应用，在车载嵌入式系统中将带来实现复杂度与性能损耗的双重挑战。

为应对上述挑战，国家“十四五”规划及《后量子密码标准化推进战略》明确指出应加快布局后量子密码基础研究与产业应用，推动关键领域率先部署抗量子通信方案。美国国家标准与技术研究院（NIST）已公布后量子密码算法标准化结果，正式确定 Kyber 为标准公钥加密算法，Dilithium 为标准数字签名算法，后量子密码算法硬件模块的性能与面积优化已成为智能汽车信息安全的关键方向。

目前，国内外研究团队正围绕 Kyber 与 Dilithium 算法的硬件加速及其在安全模块中的集成进行深入研究，并已取得阶段性成果：二者均已在 ARM mbedTLS、RISC-V TrustZone 等平台中完成了标准化实现[5][6]。**研究者主要从多项式乘法与哈希函数两方面着手优化，以提升计算效率、降低面积开销**，提升两种算法在嵌入式设备中的性能以及在资源受限环境下的部署可行性。

在针对多项式乘法的优化上，**在面积开销方面**，Shiyang He 等人[7]提出了紧凑型模块化乘法单元、压缩以及解压缩模块，资源开销较原始 FPGA 实现大幅下降；SiFan Ao 等人[8]提出多项式矩阵的逐项生成与变量重用策略以减少显著寄存器数量。**在计算效率方面**，孙俊彦等人[9]设计了一种基于 K2-RED 算法的高效模乘单元，支持 NTT、INTT 和 PWM 三种操作，其在 FPGA 平台上的性能实现完成了阶段性突破；Greconici 等人[10]在 Cortex-M 平台上实现的向量表通过预计算以显著降低总延时。

在针对哈希函数的优化上，**在面积开销方面**，Banerjee 等人在 Sapphire 平台上提出了具备一定程度模块复用能力的 Keccak 流水架构，**但哈希函数整体面积的优化无显著进展**。**在计算效率方面**，Sideris 等人在 Virtex-7 FPGA 平台上设计了基于两级子流水结构与精简轮常数生成的 Keccak-512 架构，实现了高达 36.358 Gbps 的吞吐率；Baldwin 等人提出了通过单寄存器存储状态值的实现方案，完成了 Keccak 多种变体的硬件加速。然而现有的方案难以兼顾面积与效率，在轻量化平台上存在部署困难。

除此之外，针对 Kyber 与 Dilithium 算法中如随机数据采样等其余关键算子的研究仍较为有限，尚未取得显著进展。

(3) 面向车载平台的格密码硬件架构

综合已有研究，团队认为 Kyber 公钥加密算法与 Dilithium 数字签名算法在车联网级嵌入式设备上的协同部署**主要受面积资源限制**，其面临的主要困难可归结如下：

（1）格基密码算法的主要运算——多项式乘法运算会产生大量中间数据并且计算复杂度相对较高，对存储空间和计算单元需求量大且涉及复杂寻址，独立部署两算法将显著增加存储压力；

（2）Kyber 公钥加密算法与 Dilithium 数字签名算法均包含多种随机采样过程且采样规则繁杂，对寄存器资源需求量大，影响整体面积利用率；

（3）Keccak 哈希函数的海绵结构带来并行化的困难，其优化难以兼顾计算效率与面积开销，现有的优化方案不适合于面积受限的车联网平台。

针对上述问题，本作品提出了一种可共享通用算子、兼容 Kyber 与 Dilithium 算法的二合一硬件体系结构。该体系以模块集成和动态调度为核心机制，通过统一调度器与共享内存映射实现面积与性能的双向优化。具体而言，作品在哈希计算、随机采样与多项式运算三个关键环节中进行了结构级改进：

1. 重构高吞吐的哈希模块。针对 Keccak 置换的性能瓶颈问题，设计采用“双阶段迭代 + 子步骤重组”的细粒流水优化策略。通过将五步子步骤分解变换为四步，减少一次中间存储与数据依赖，在优化消息的处理速度的同时降低存储需求；同时构建两级子流水结构，提升了哈希算法的吞吐率。

2. 提出统一标准的采样模块并进行资源共享。面向多样化采样规则，建立统一采样路径将采样逻辑划分为四级标准模块。其中三层模块（缓冲、提取、存储）通过寄存器复用实现资源共享，核心映射层根据控制信号独立切换模式与读取位宽，该结构在保持三种采样方式完整功能的同时，简化控制信号种类约 67%，并通过消除重复寄存器显著降低面积冗余。

3. 设计统一的多项式运算架构。在存储层引入 4-Bank 并行结构与动态混合进制地址映射，实现零冲突访存，提高访问效率与数据吞吐。在计算层，利用旋转因子的对称性和周期性共享特征，统一两算法的乘法流程，并构建双 ALU 解耦路径以支持多类型乘法运算。该设计通过实现两种算法计算逻辑的融合以显著降低面积冗余。

通过以上结构优化，系统在资源受限平台上实现了后量子加密与签名算法的协同运行，兼顾面积效率与计算性能，为车载安全芯片提供了可扩展的硬件实现路径。

1.2 作品特色

在解决上述问题的基础上，本作品同时具备如下创新特色。

1. 安全种子逐级派生机制。系统以 DRAM-PUF 输出作为主随机数种子，通过哈希函数分层派生生成各采样环节所需的子随机数种子，实现按需动态生成与更新。该机制保证了随机种子无需从外界读取，在生成与使用全过程中均不以明文形式存储或

传输，从而有效降低潜在泄露风险。同时，由于无需长期保存多级种子，系统整体的存储开销得到显著压缩，提升了随机数管理安全性与资源利用率。

2. 规模可变的最优算子结构。针对不同算子在计算效率与面积占用之间的权衡关系，系统预先建立多种可选硬件实现方案，并对各方案的性能指标进行量化建模。在此基础上，将资源受限的车联网级平台的总面积作为约束引入优化目标函数，构建以总面积最小化为约束、计算效率最大化为目标的规划求解模型。通过对该模型的求解，可在限定资源条件下确定算子级别的最优配置，实现系统整体计算效率的全局最优化。此外，该模型使得本共享架构具备可迁移至其他规模嵌入式平台的能力。

3. 基于动态地址映射的访存冲突简化机制。针对多算子并行执行过程中存储访问冲突频发的问题，系统设计了基于动态地址映射与优先级仲裁的统一访存管理机制。通过建立 Bank–Index 地址映射表，对多算子共享的数据块进行层次化编号与重映射，使不同运算单元在同一周期内能够访问互不冲突的存储区。同时，引入自适应访问仲裁逻辑，根据算子运行状态动态调整读写优先级，从而平衡数据带宽分配并减少总线等待时间。该机制有效降低了访存冲突带来的管线阻塞，简化了寻址控制逻辑，并在保证访存一致性的前提下提升了系统带宽利用率与整体计算效率。

综上所述，本作品的研究成果与《智能网联汽车创新发展战略》及《工业互联网创新发展行动计划》等国家政策高度契合。所提出的二合一后量子密码硬件体系为车载信息下的数据安全与身份认证提供了可靠的保障，同时具备良好的与现行密码服务的兼容能力，为我国在该领域的硬件标准化与产业化奠定了技术基础，具有重要的工程应用价值与战略意义。

在未来工作中，本作品提出的融合型硬件架构将继续沿设计优化与安全增强两个方向深入发展。系统将针对计算效率、可扩展性及抗侧信道攻击能力开展进一步优化研究，并探索在异构平台上的可移植实现，以构建兼具高性能与高安全性的通用后量子密码硬件基础。这一方向的持续推进，有望为智能汽车、物联网及关键基础设施领域提供可验证、可部署的后量子安全支撑体系，助力我国数字化与网络安全体系的长期发展。

第二章 背景知识

2.1 密码算法介绍

2.1.1 格密码

随着密码分析技术与量子计算的发展，传统公钥密码体系面临严峻的安全挑战，基于更复杂的代数结构设计的后量子密码 (PQC) 算法应运而生。当前主流方案包括基于格、编码、哈希及多变量多项式四类。PQC 算法以格密码在工程实现上最为成熟，其中典型代表 Kyber 密钥封装算法与 Dilithium 数字签名算法已入选 NIST 标准。

较于其他候选的后量子加密算法，Kyber 与 Dilithium 拥有更优秀的计算性能和更成熟的生态支持，如图 2.1所示，两种算法在内存占用、实现复杂度等方面表现出良好的综合优势，被广泛认为是面向通用平台与嵌入式系统最具实用性的组合。同时，二者均基于格理论中的 MLWE 难题，且均以 Keccak 哈希函数作为伪随机数生成方案、以多项式商环乘法作为核心运算，在具备进行模块整合从而在嵌入式系统中协同部署潜力的同时也存在大量可优化空间，图 2.2以密钥生成算法为例展示了二者的共享算子。

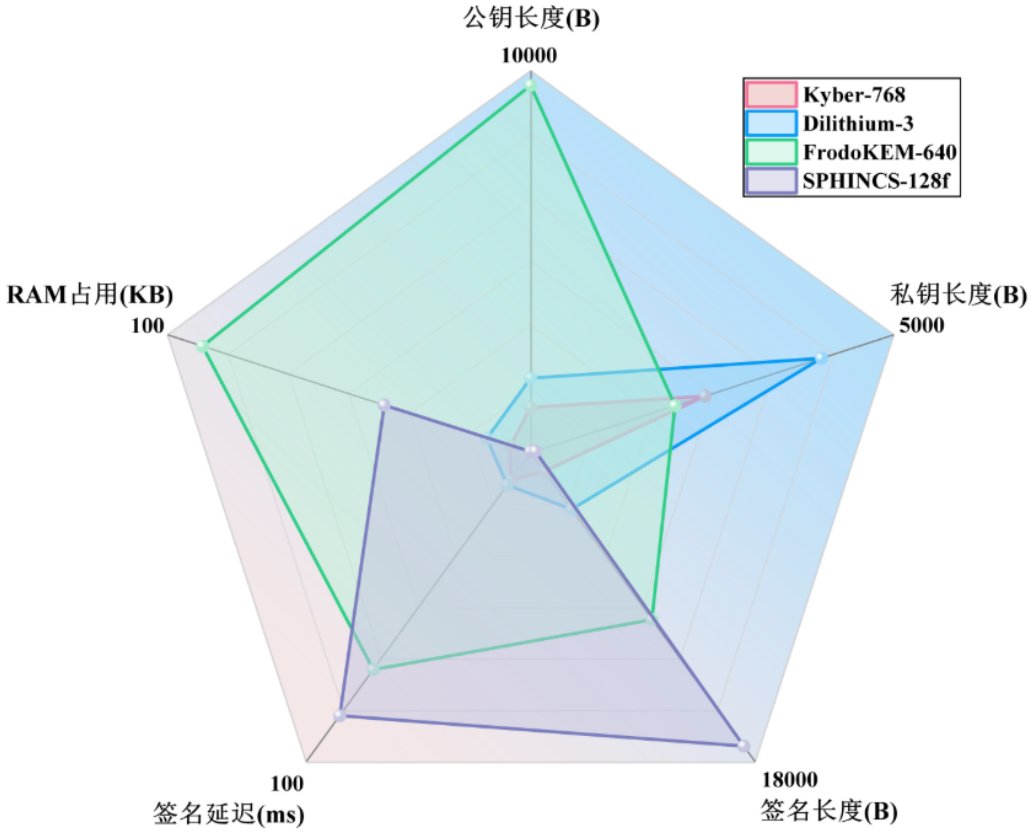


图 2.1　后量子密码算法的性能对比

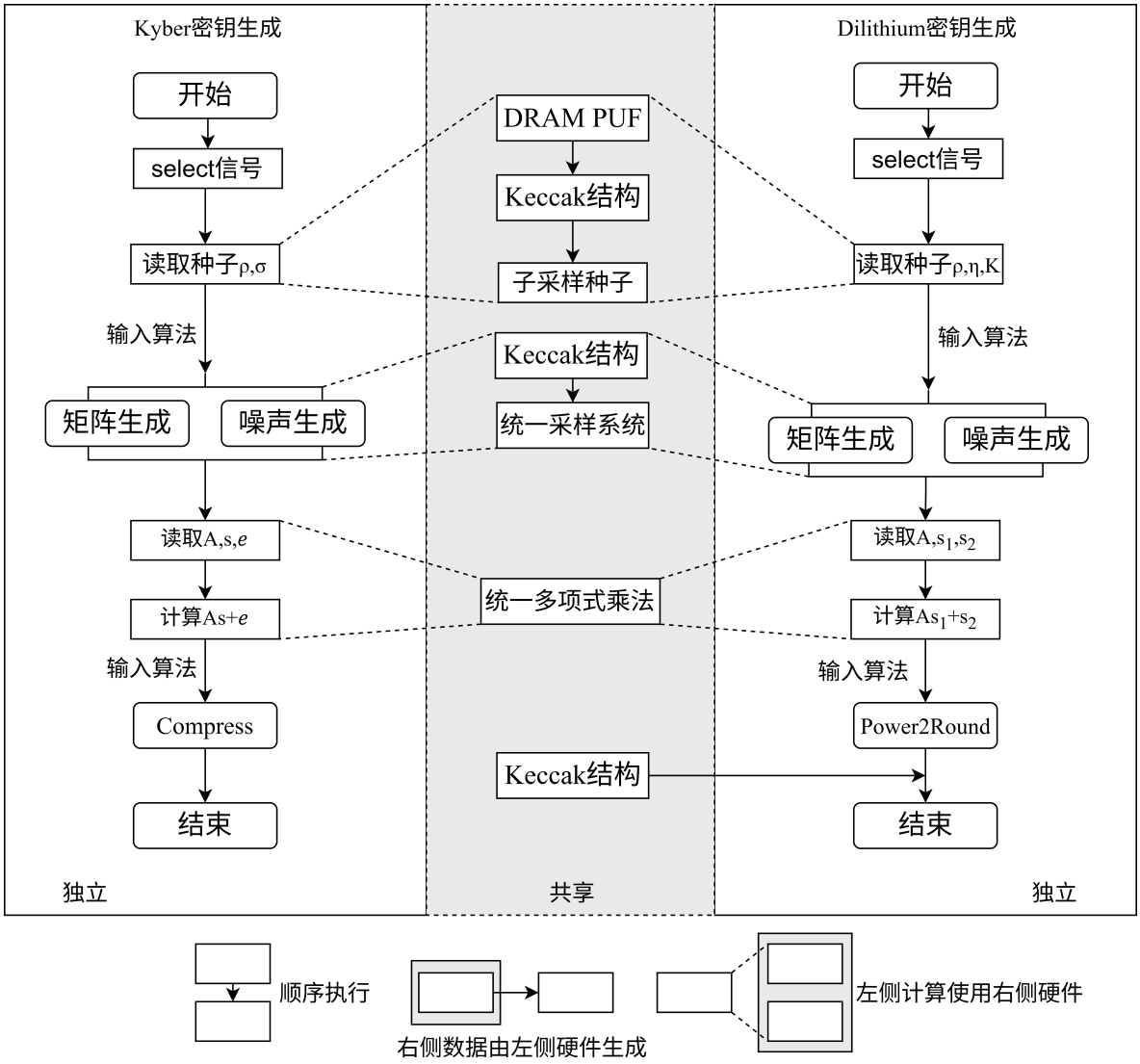


图 2.2　密钥生成算法的共享算子

2.1.2 Keccak 函数

SHA 系列算法作为哈希函数，迄今为止一直在不断更新迭代以应对各种新的安全威胁，Keccak[11] 凭借使用海绵结构和固定置换函数的独特设计被采纳为应对量子威胁的标准，其具有高度的安全性，在数据压缩与伪随机数生成等密码学应用领域中发挥重要作用。

Keccak 基于海绵结构设计，具备可扩展输出长度、强安全性与参数可调性，适用于多种场景。其运算分为吸收和挤压两个阶段，在开始前首先对消息进行填充，在其后添加开头结尾均为‘1’中间为‘0’的比特流，使长度为比特率 r 的整数倍。填充后的消息被划分 n 个长度为 r 位的消息块 M_i 送入结构进行迭代。

在吸收阶段，首先将内部状态 S 初始化为全零比特串，随后每次吸收一个消息

块 M_i，将其与 S 的前 r 比特逐位异或，然后对 S 应用置换函数 Keccak-f[b] 更新。该过程不断重复，直到所有消息分组被吸收。之后进入挤压阶段输出散列值。每次从状态 S 中提取前 r 比特作为输出，随后应用置换函数 Keccak-f[b] 更新状态并准备下一次输出，重复该过程直到累积的输出达到所需的长度 d。当 d 不是 r 的整数倍时，在最后一次挤压时截断处理以适配标准。

在上述参数中，比特率 r 表示每轮处理的数据量，容量 $c = b - r$ 控制抗碰撞能力，当 $c \geq 2d$ 时，可抵御 d 位安全攻击。SHA-3 中状态宽度固定为 $b = 1600$，对应不同 r 值展现 Keccak 在输出长度与安全强度间的灵活适应能力。

2.2 多项式商环乘法的优化理论

在 Kyber 与 Dilithium 密码算法中，多项式商环乘法是其涉及的主要计算，直接关联整体性能。该类运算在硬件实现中复杂度高、元件开销大，本节将围绕模约简方法与基于数论变换 (NTT) 的多项式乘法优化展开探讨，为后续优化实现奠定基础。

2.2.1 模约简和模乘法的优化

模约简 $z = x \bmod q$ 是密码算法的必要运算。传统除法器的复杂度为 $O(n^2)$ 且硬件实现开销大，难以适配资源受限场景；为此 Plantard、Barrett 以及 Montgomery 等基于重构和预计算的方法 [12] 被提出，有效地提升了在硬件上模约简的效率。

Plantard 算法是一种适用于结构化模数 $q = 2^k + c$（$c << q$）的模约简加速方法，其将模约简转换为移位和掩码操作，适用于 $x \in (0, 2q^2)$ 的情况，图 2.3展示了其逻辑电路结构。

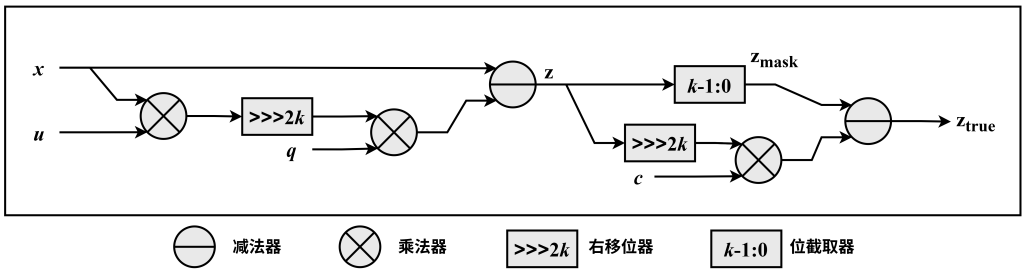


图 2.3　Plantard 模约简逻辑电路

Barrett 算法的执行流程与 Plantard 算法相似，二者的不同之处在于 Plantard 算法对模数 q 的形式有一定要求，但由此带来了计算上的简便性，而 Barrett 算法不需对其作要求，但需额外执行条件判断来确认输出值是否符合结果，适用于 $x \in (0, q^2)$ 的情况，图 2.4展示了其对应的逻辑电路结构。

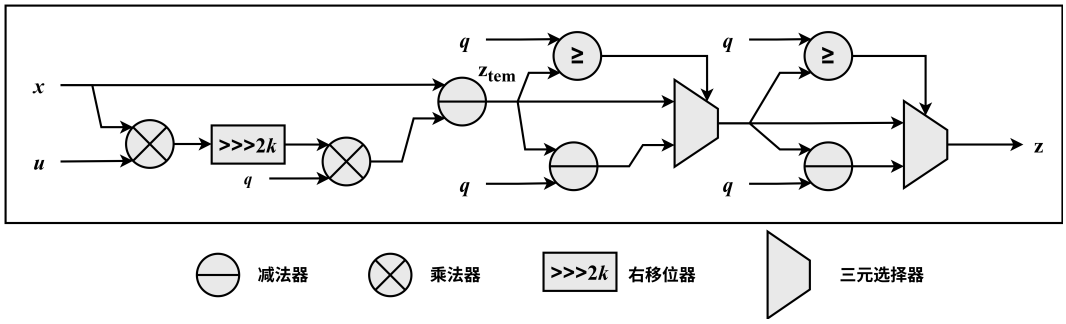


图 2.4　Barret 模约简逻辑电路

Montgomery 算法用于有限域上的乘法 $ab \bmod q$ 计算，能够将运算转换到 Montgomery 域以减小尺寸，大规模减小所需的存储空间，并通过乘法和移位代替除法加快计算。其核心步骤分为 REDC 映射、Montgomery 域中的模乘运算以及逆 REDC 映射三个阶段。REDC 的逻辑电路如图 2.5所示。

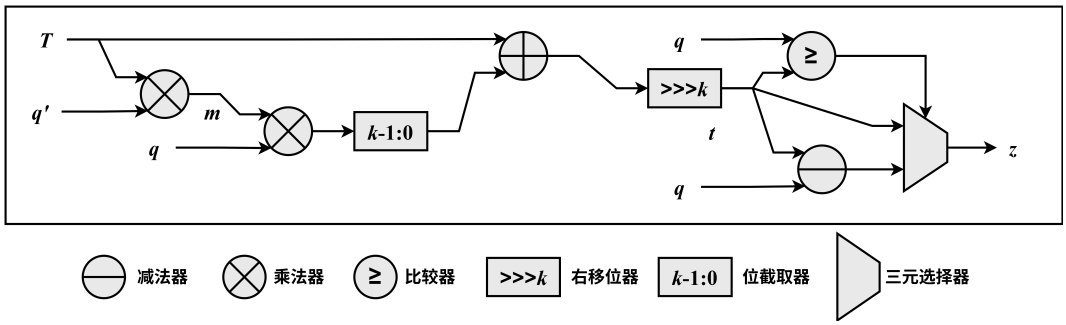


图 2.5　REDC 逻辑电路

当元素 a 和 b 在运算中可能多次出现时，Montgomery 算法仅在预映射执行一次小规模模运算，之后进行乘积计算时仅涉及移位和截取操作。因此，不仅降低了存储和计算开销，在多次混淆场景下还可显著提升整体运算效率。

对于 Kyber 和 Dilithium 两种格密码，前者的模数为 $3329 = 2^{12} - 767$，适用 Barret 算法；反之后者的模数为 $8380417 = 2^{23} - 8191$，采用 Plantard 算法更合适。在以上的基础上，再利用 Montgomery 算法简化乘法运算，提高密码算法的执行效率。

2.2.2 基于 NTT 的多项式乘法优化

多项式商环上的乘法运算操作繁杂，是格密码的硬件实现上最耗费资源的模块。传统计算方式复杂度为 $O(n^2)$，不适合计算高维多项式，根据硬件自身的特点可以设计优化算法以提高效率，其中的主要代表是复杂度为 $O(n \log n)$ 的数论变换 (NTT)[13]。

一个 (n-1) 次多项式具有 n 个系数，如果已知该多项式上 n 个不同的点值，便可以唯一确定该多项式。若多项式 $f(x)$ 和 $g(x)$ 均已知对应 $2n$ 个的点值，那么可以得到 $f(x) * g(x)$ 在对应点上的取值。若能够高效完成多项式在特殊点的取值与系数间

的相互转换，即可快速计算多项式的乘积，这就是 NTT 的核心思想。然而遍历计算多项式的每个点值仍然有较高的复杂度，利用蝶形单元构成网络可以通过将多项式进行分治计算以减少计算次数，从而将其复杂度显著降低。

定义 2.1 蝶形单元：蝶形单元由一个预设旋转因子 c 和两个输入实数 a, b 组成，输出两个实数，根据输出的结果不同可以分为具有 CT 或 GS 结构的蝶形单元。CT 结构的输出分别为 $a + bc$ 和 $a - bc$，GS 结构的输出分别为 $a + bc$ 和 $(a - b)c$。

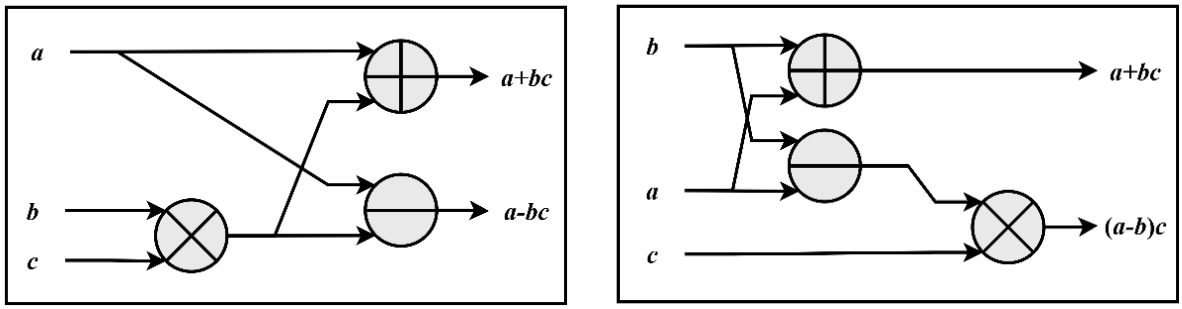


图 2.6　CT 蝶形单元电路（左）和 GS 蝶形单元电路（右）

两种蝶形单元的逻辑电路如图 2.6 所示。为简化表达，一般将二者不加区别地表示。在一个蝶形网络中只会用到一种结构，依据输出结果即可辨认。

设有 8 系数多项式 $f(x) = a_0 + a_1x + \cdots + a_7x^7$，记其 NTT 映射的点值向量为 $(A_0, A_1, \ldots, A_7)$，其各系数可以按照如图 2.7 中 CT 结构蝶形网络求得。图中 w_t 代表 t 次单位原根，各层中 t 的取值不同，每一个蝶形单元中以 w_t 的幂作为旋转因子。按正序输入 8 个系数即可计算得全部 8 个点值。图 2.8展示了进行 8 点逆操作 INTT 映射时采用的与 NTT 对称的 GS 蝶形网络，其各层使用 w_t^{-1} 的幂生成旋转因子且结构与 CT 网络相反。

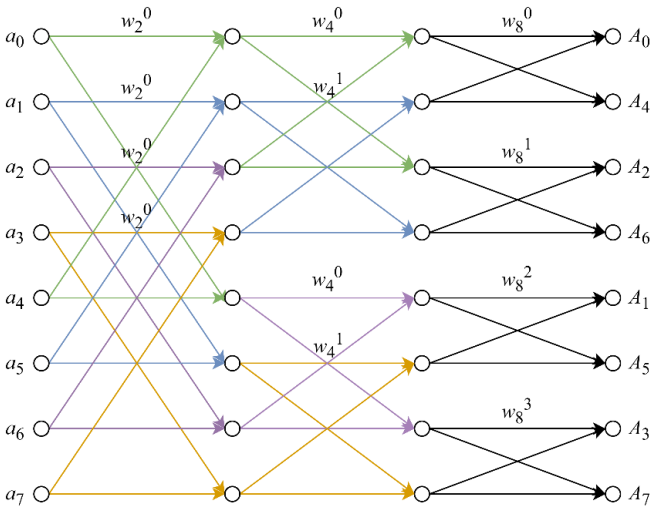


图 2.7　8 系数 CT 蝶形网络

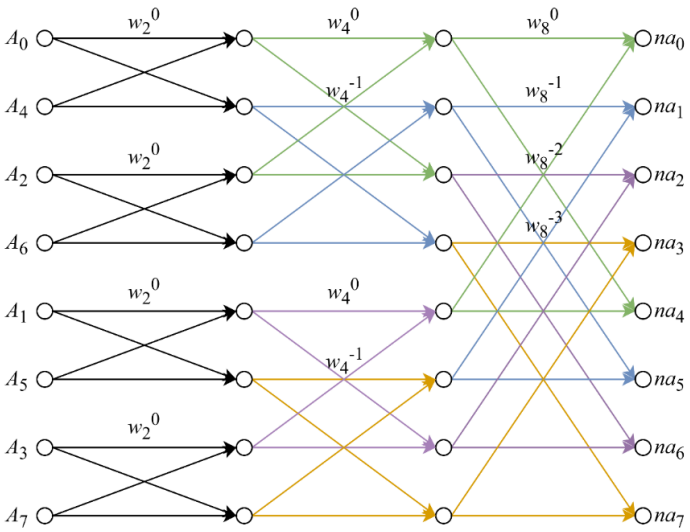


图 2.8　8 系数 GS 蝶形网络

在此基础上，还可利用 Negative Wrapped Convolution NTT (NWC NTT)[14] 技术隐性消除对最终乘积结果的模多项式运算。通过以上优化方式，多项式乘法在保证计算正确性的基础上实现了运算效率与硬件资源占用的双重优化，为核心计算逻辑的高效硬件实现奠定了基础，然而如何将此类优化算法在具备具体规模与性能的嵌入式设备，则仍需具体发掘。

第三章 核心技术实现

3.1 多级种子管理机制

在 Kyber 密钥封装算法与 Dilithium 数字签名算法中，伪随机数据流均由固定长度的随机数种子经哈希函数映射所得，其安全性与随机数种子的安全性息息相关。传统方案依赖预存储各随机数种子或外部输入，存在存储安全隐患，且种子管理复杂。

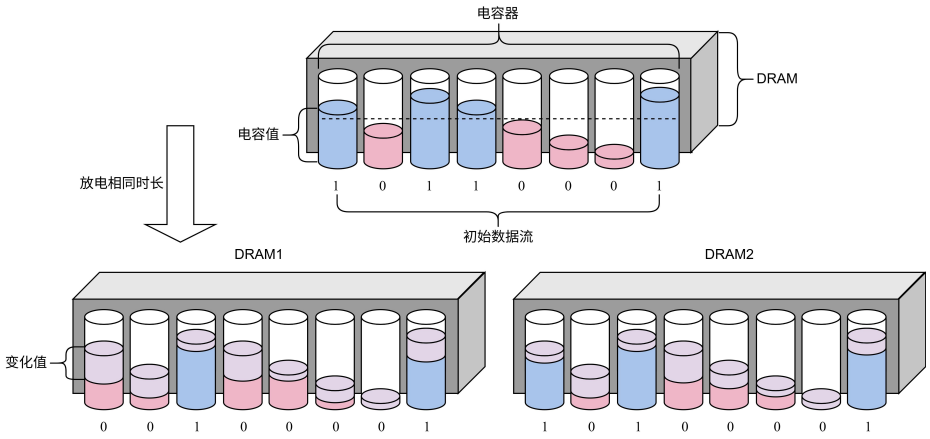


图 3.1　DRAM PUF 的工作原理

为提升整体安全性并降低种子管理难度，本作品基于物理不可克隆函数（PUF）设计了多级种子管理机制。采用 DRAM PUF[15] 作为主种子发生器（如图 3.1所示），其原理基于存储单元在放电过程中电荷衰减速率的天然差异，致使在相同初始化状态和放电时间下同一设备的不同单元以及不同设备的存储阵列的电荷衰减速率均不相同。据此能够生成具有强随机性的数据流作为主种子，并结合哈希函数生成子种子。

定义主种子 S_0 与各子种子 S_i 生成过程为：

$$S_0 = PUF_{DRAM}(V_{\text{ref}}, T), \quad S_0 \in \{0,1\}^{256}$$

$$S_i = HASH_{SHA3}(S_0 || ID_i)$$

密码算法中各子种子的信息如表 3.1所示，具体生成按照如下所述方式进行。

Step1：设定所有 DRAM 的单元电压的为 V_{ref}；

Step2：关闭 DRAM 的刷新控制，执行放电过程 T，读取之后电荷状态形成 T_0；

Step3：读取所需子种子标识 ID_i，执行 $S_i = \text{HASH}_{\text{SHA3}}(S_0||ID_i)$。

种子生成结束后，可重新初始化 DRAM 重复执行上述流程，其执行逻辑如图 3.2所示。为降低安全风险，系统允许定期更换初始状态 V_{ref}。

表 3.1　Kyber 与 Dilithium 中的子采样种子

种子 ID	位宽	所属算法	所用算法
ρ	256 bit	Kyber	矩阵生成算法
σ	256 bit	Kyber	噪声生成算法
r	256 bit	Kyber	噪声生成算法
ζ	256 bit	Dilithium	哈希函数
χ	348 bit	Dilithium	掩码生成算法

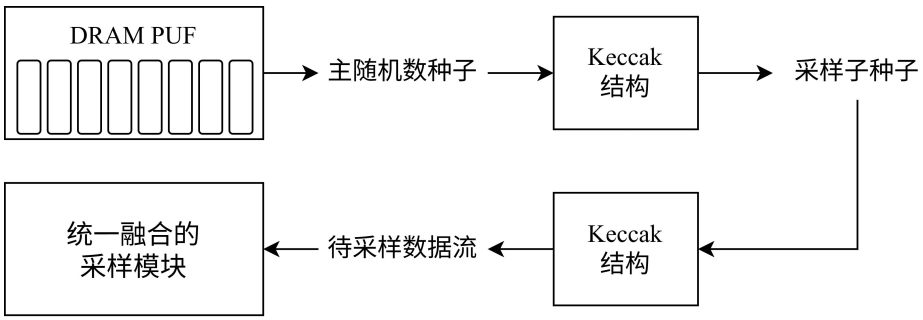


图 3.2　子采样种子的生成原理

该机制仅需一次 PUF 唤醒即可生成主种子，并通过拼接不超过 3 位的唯一标识扩展生成采样种子。与传统显式存储相比，存储需求由至少 1372 比特减少至 256 比特，同时简化了寻址与控制逻辑。

3.2 基于硬件的重构哈希加速器

Keccak 函数作为统一哈希模块核心，承担着密钥派生与伪随机数生成等关键计算任务。本作品设计如图3.3所示，实现由主 CPU 通过访问寄存器实现数据写入、计算发起和结果读取，可以在公共缓冲区完成与采样器的交互。

架构聚焦于 Keccak 模块在高并发与低延迟处理中的双重性能困境：一方面需提升整体并发吞吐能力以支撑多消息批量处理，另一方面需优化单条消息的执行效率与时序表现，在**置换函数核心模块**上，分别设计**双阶段流水的细粒调度架构**与**子步骤级重构架构**解决相应痛点：

（1）双阶段流水的层级调度架构（Dual-Stage Round Pipeline，DSRP）：本架构将五步置换划分为 F1（θ→ρ→π）和 F2（χ→ι）两段流水，打破传统轮函数“单一入口–单一出口”的串行限制；设计了支持常规数据路径与冻结-恢复路径的双模式的智能状态机，实现对阶段内任务的统一调度与资源管理。借此，系统有效缓解多任务调度瓶颈，达成高吞吐并发处理。

（2）子步骤级融合重构（Folding-Two-Step，FTS）：基于对五步置换运算特性的深度分析，本架构提出以硬件融合为核心的重构策略，具体包括：

- **合并绕线法（合并 ρ 与 π 步骤）**：计算适配的预目标位置与旋转偏移量，将两个步骤融合为单一布线操作，实现零逻辑开销的位重排；
- **资源压缩法（θ、χ、ι 步骤）**：利用硬件 LUT 资源将多级组合逻辑转化为单级查表操作，降低 θ 与 χ 步骤关键路径延迟与资源消耗；ι 步骤仅对 7 个特定有效位计算，采用稀疏存储与按位索引机制，显著减少异或操作资源消耗。

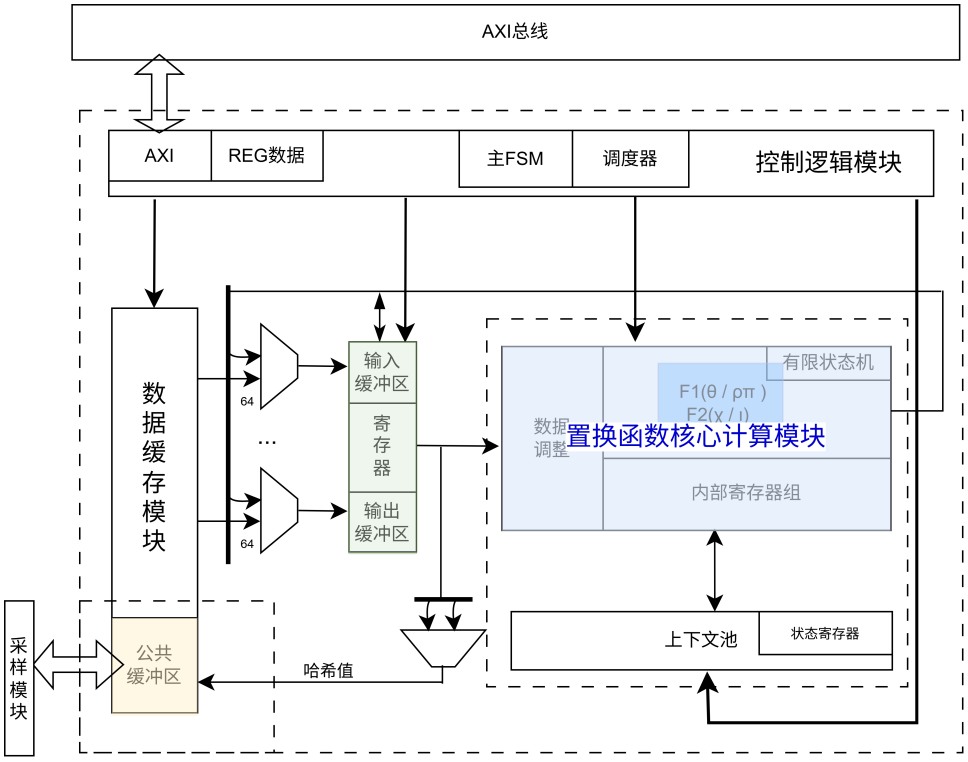


图 3.3　Keccak 设计总架构

3.2.1 双阶段流水的层次调度架构

在基本迭代架构上，文献 [16] 指出二级流水线结构是在吞吐能力提升与面积资源之间的最佳折中点。为适配车载需求，本作品设计双阶段轮流水线架构，如图3.4所示对 Keccak 五步置换进行了关键性重构，将其划分为前段 F1（θ, ρ, π）与后段 F2（χ, ι）两个流水阶段，使得 F1 专注于状态的线性变换与位置重排，而 F2 则执行非线性的位运算与轮常数扰动。配合设计的并发任务调度机制，可以使系统在严格保持算法逻辑顺序的前提下，实现多条消息的动态交错执行。在此模式下，单时钟周期内可有多达四条数据块分别处于不同的执行状态，构成高效的“前向-回绕”调度流程，硬件资源利用率提升至传统串行架构的 4 倍。

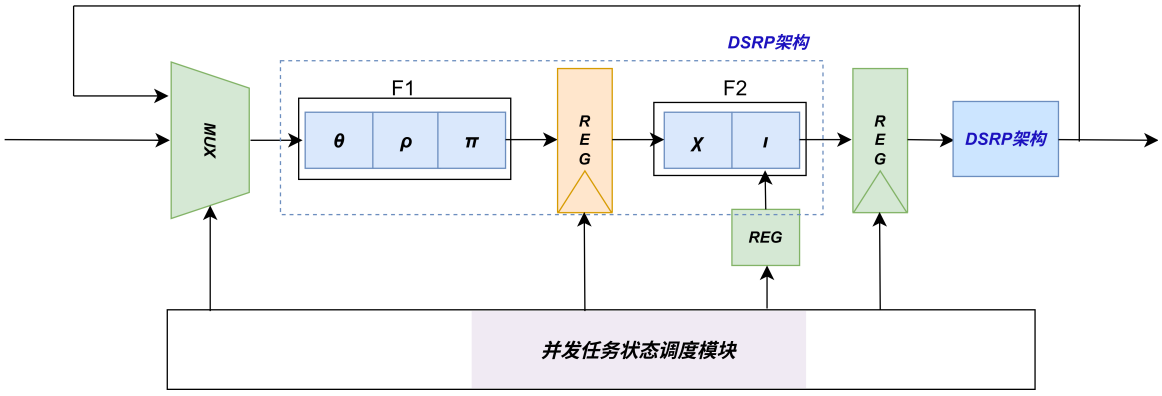


图 3.4　DSRL 架构流水线实现流程图

其中，层次并发任务状态调度机制的关键组成如图3.5所示。该机制以**统一主状态机**为最高控制核心，协调关键模块：**多消息调度器**作为决策中枢，负责依据优先级策略动态分配计算资源；**状态仲裁器**实施挤压优先与延迟冻结策略，确保吸收与挤压阶段的资源互斥执行；**上下文池**作为系统的全局状态记忆库，集中维护所有活跃任务的进度与阶段信息；**状态冻结模块**和**逻辑控制模块**则分别负责任务的挂起与恢复流程调度、将抽象调度决策转化为对计算核心的精确控制信号。

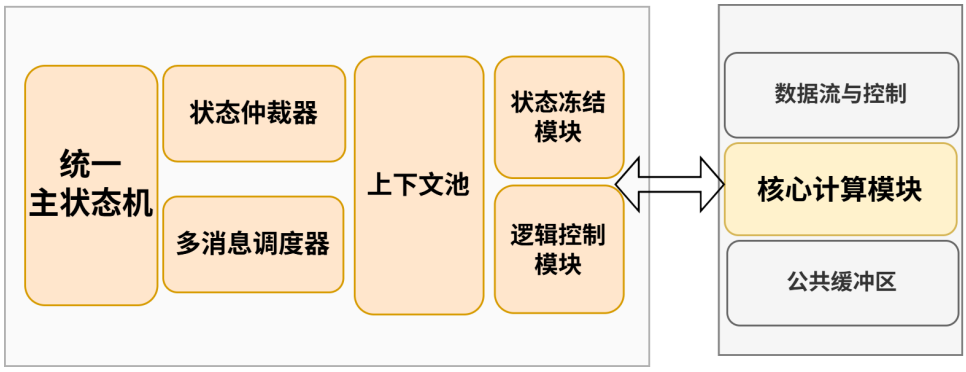


图 3.5　并发任务状态调度结构

(1) 统一主状态机

统一主状态机实现常规数据路径与冻结-恢复路径的双模式智能调度的调度,具体设计如图3.6所示,该主状态机构建了完整的“吸收-挤压-恢复”调度闭环。常规数据路径构成了系统处理的基本循环:从检测任务就绪的 $IDLE$ 与进入队列的 $CHECK_FIFO$ 状态，到进行运算的 $ISSUE_F1$ 状态，最终进入等待操作完成的 $WAIT$ 状态，形成一个持续吸纳与处理消息的高效流水线。

在上述基础循环外, 面对挤压请求, 状态机仍具备精细化调度能力。当仲裁器检测为第一轮挤压时,系统仅经 FREEZE 状态完成轻量同步后便立即返回 $CHECK_FIFO$, 几乎不影响吸收任务的持续调度; 而对于需要执行置换的多轮挤压，状态机则启用 $SQUEEZE_EXEC$ 状态，专用于处理耗时的挤压置换计算。此设计使得长时间的

资源独占操作被隔离在特定状态中，避免阻塞常规调度逻辑。

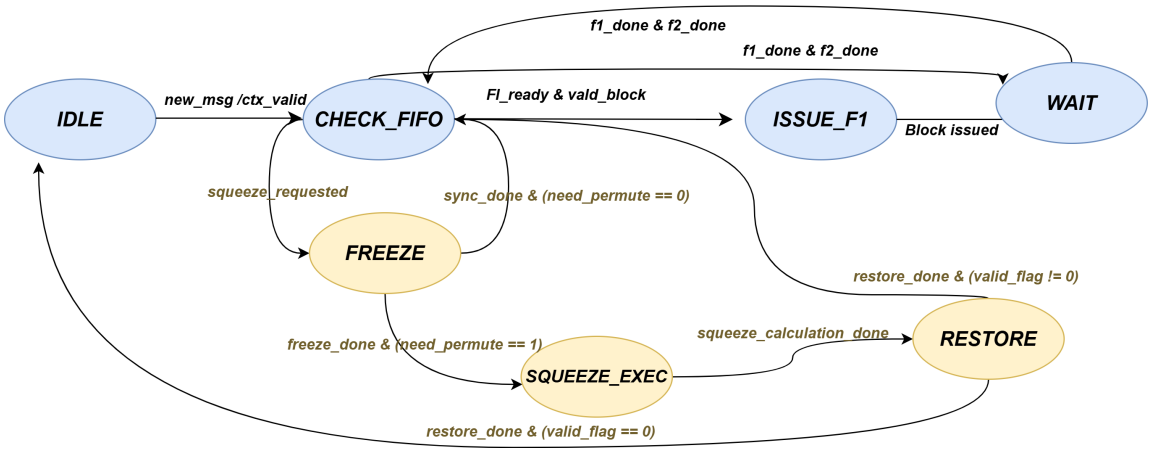


图 3.6　统一主状态机

挤压计算完成后，状态机进入 RESTORE 状态，依序恢复此前冻结的吸收任务。恢复完成后,系统根据全局任务状态智能决策:若仍有活跃任务,则返回 $CHECK_FIFO$ 继续调度；若所有任务均已完成，则回归 $IDLE$ 空闲状态。

(2) 冻结调度的核心设计

上述统一主状态机为系统提供了清晰的调度框架，而冻结调度机制则兼顾了系统稳定性和性能优化的平衡，包括两个核心环节：安全冻结与有序恢复。

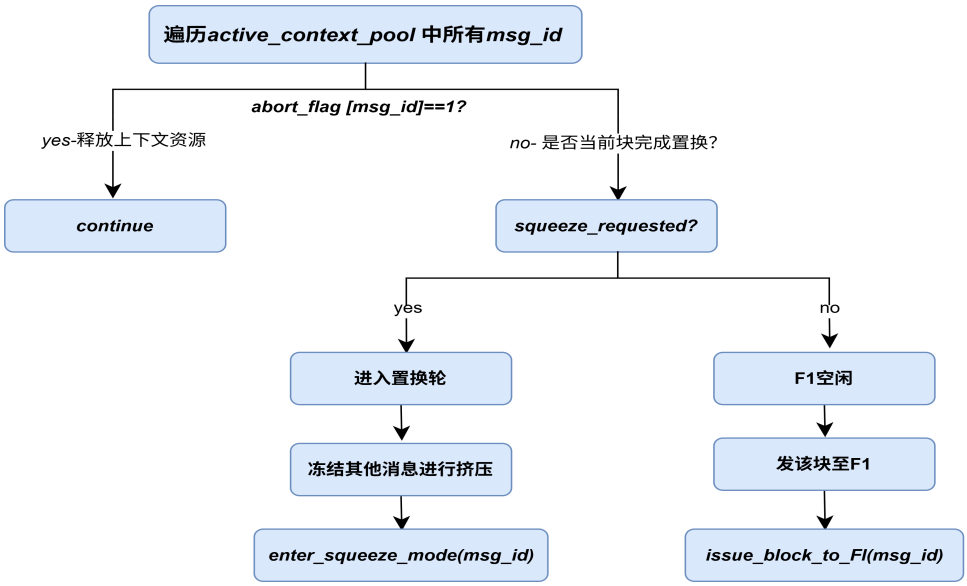


图 3.7　多消息调度器的核心工作流程

冻结操作仅在吸收阶段数据块完成完整的五步置换后触发，确保状态完整性。由于设计流水线并行度为 4，最多仅需等待一个时钟周期即可完成安全冻结，主要通过多消息调度器配合完成信息块工作。

其核心流程如图3.7所示，采用基于优先级的动态决策机制，每周期对系统中的

所有活跃消息进行状态评估与资源分配。调度器首先检测异常终止请求，及时释放被占用资源；随后响应挤压任务需求，通过冻结机制确保关键计算资源的可用性；最后调度就绪的吸收任务，维持系统的高吞吐处理能力。这种分层次的调度策略在保障功能正确性的同时，最大限度地提升了硬件资源的利用效率。

而在恢复操作中，冻结状态调度采用 FIFO 队列与轮转调度相结合的策略，周期检测 F1 资源可用性，最多允许两条冻结消息同时恢复。如算法3.1所示，若任务处于就绪状态且系统有空闲计算资源，则立即将其分配至执行单元并更新任务状态；若任务仍在等待前置操作完成，则持续追踪其状态。该恢复策略有效避免了资源拥塞，确保系统从冻结状态平滑过渡到全速运行。

Algorithm 3.1 冻结任务恢复调度算法

```
for each slot a in freeze_queue do
    if msg_id[a].state == READY and F1_free > 0 then
        F1_unit[F1_free-1] ← freeze_queue[a].msg_id
        context[freeze_queue[a].msg_id].state ← RUNNING_F1
        F1_free ← F1_free - 1
        freeze_queue[a].valid ← 0
    else if msg_id[a].state == WAIT_F2 then
        Wait for F2 completion
    else
        Remain pending
```

双阶段流水的细粒调度架构在确保功能正确性的同时，实现了资源利用率和系统吞吐率的最大化。最终，该架构在资源受限配置下实现稳定吞吐率达 6.71 Gbps，相比传统架构方案，在资源利用率与调度效率上均实现显著提升。

3.2.2 子步骤级融合重构 FTS

上文提出的多消息流水线架构有效提升了系统整体吞吐能力，本节基于五步置换的运算特性与数据依赖结构基础，对于单条消息的处理延迟与逻辑资源效率进行优化，提出对应的子步骤级融合重构架构——Folding-Two-Step(FTS) 架构。其核心在于发掘置换中存在的硬件优化机会，特别的，通过步骤硬件融合得到预计算目标变体参数表，在保证功能等效的同时，实现了近零成本的性能跃升；此外，利用硬件资源将复杂布尔逻辑转化为单级查表、避免无作用运算流程和资源消耗。

上述架构基于 Keccak-f[1600] 实现，该方法的参数化架构支持适配其他 Keccak

变体。由于各子步骤在 z 维度上均为逐比特独立操作，无跨 z 维数据依赖。于是，基于硬件特性可将每组 (x, y) 直接打包封装为单一 64bit 宽度字（word），实现 z 维度的全位并行，映射为二位字阵列 $[63:0]$，为后续优化提供统一的硬件基础。

(1)ρ-π 步骤硬件融合

传统实现中，ρ 步和 π 步作为两个独立步骤顺序执行。经观察，发现两者均可视为静态的确定性映射操作，运算过程不存在数据相关性。基于该特性，如图3.8所示，本架构设计合并绕线法，将两部分在硬件层面融合为统一的纯布线操作。

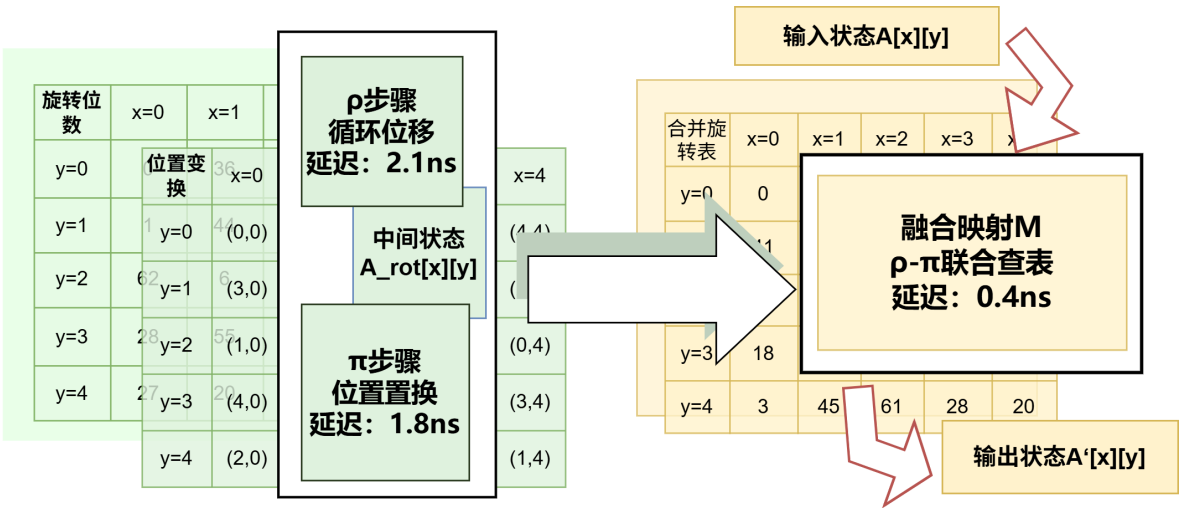


图 3.8　两步融合优化架构

本架构下，Keccak 状态空间为 $S = \mathbb{F}_2^{5\times5\times64}$，定义两个映射函数表示旋转映射和置换映射：$\mathcal{R} : S \to S$：$\mathcal{R}(A)[x, y] = \mathrm{ROTL}(A[x, y], \delta(x, y))$，$\mathcal{P} : S \to S$：$\mathcal{P}(A)[x, y] = A[(x + 3y) \bmod 5, x]$，其中 $\delta : [0, 4]^2 \to [0, 63]$ 为预定义的偏移量函数。由于两者不存在数据相关性，本团队设计融合映射 $\mathcal{M}$，使满足 $\mathcal{M} \equiv \mathcal{P} \circ \mathcal{R}$，有 $\mathcal{M} : S \to S$：$\mathcal{M}(A)[x, y] = \mathrm{ROTL}(A[\pi_1(x, y), \pi_2(x, y)], \delta'(x, y))$，其中 $\pi_1(x, y) = (x + 3y) \bmod 5$ 为源 x 坐标；$\pi_2(x, y) = x$ 为源 y 坐标；$\delta'(x, y) = \delta(\pi_1(x, y), \pi_2(x, y))$ 为融合偏移量。

在硬件实现中，$\mathcal{M}$ 便可直接映射为纯组合逻辑，具体代码如下所示。

```
// 融合映射的硬件实现

module fused_rho_pi #(parameter  1, parameter  2, parameter  ') (
    input [63:0] A_in,
    output [63:0] A_out
);
    assign A_out = {A_in[63- ':0], A_in[63:64- '+1]};
endmodule
```

在顶层模块内部，系统依据查表结果批量实例化 25 个独立的 $fused_rho_pi$ 子模

块，完成整个 Keccak 矩阵的 ρ–π 融合映射，代码如下所示。该方式通过参数化配置实现统一逻辑模板复用，使得整体逻辑无任何动态控制或逻辑门参与，所有路径均在综合阶段被工具自动优化为纯绕线映射，逻辑延迟趋近于零，资源消耗极小。

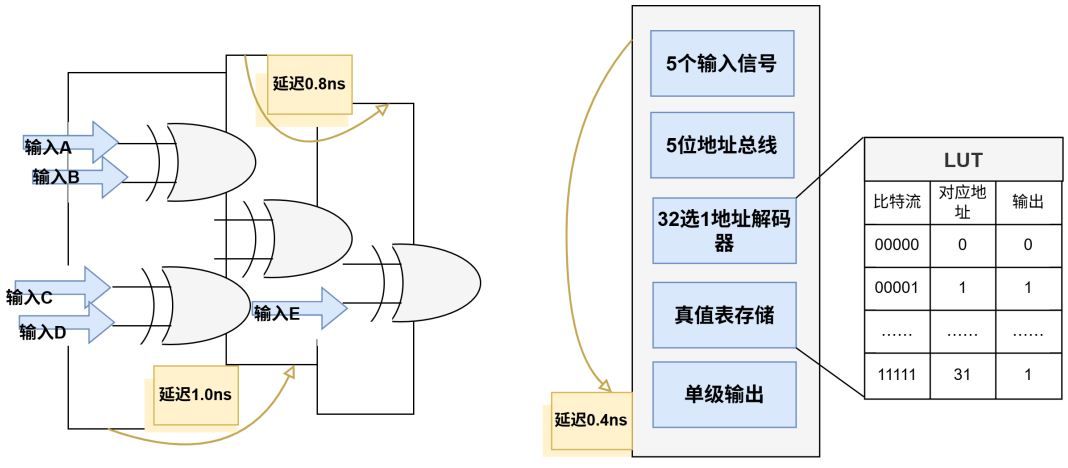
```
//逻辑模板复用的统一参数化配置

module keccak_rho_pi (
    input wire [63:0] A[0:4][0:4],
    output wire [63:0] A_out[0:4][0:4]
);
    // 根据之前生成的映射表自动连接
    fused_rho_pi #(.SRC_X(0), .SRC_Y(0), .SHIFT(0)) u_00 (.A_in(A[0][0]),
        .A_out(A_out[0][0]));
    fused_rho_pi #(.SRC_X(1), .SRC_Y(1), .SHIFT(44)) u_10 (.A_in(A[1][1]),
        .A_out(A_out[1][0]));
endmodule
```

该融合优化通过预计算的布线路径，在硬件开销和时序性能方面展现出显著的量化优势。基于 Xilinx Artix-7 FPGA 平台的综合实测数据，融合后资源利用效率提升超过 10 倍，实现了近 91% 的逻辑资源节省；同时关键路径延迟降低了近 90%，为系统时钟频率从约 256MHz 提升至 2.5GHz 提供了理论可能。

(2) 其余子步骤的逻辑压缩

在核心融合优化上，该架构同时基于硬件特性对其余子步骤进行优化，主要在于优化长逻辑深度、避免无效运算流程。



图(a) 逻辑门级联法　　图(b) 存储器查找法

图 3.9　多级逻辑硬件实现方式对比

θ 与 χ 步骤分别进行线性异或计算、非线性布尔映射。以 θ 步骤为例，如图3.9所

示，主要有逻辑门级联法和存储器查找法两种硬件实现方式。本作品采取存储器查找法，将 θ 步骤的 5 输入异或树与 χ 步骤的非线性布尔逻辑转化为单级查表操作。比较发现对于 θ 步骤，传统实现需构建 3 层 XOR 级联树，逻辑深度高达约 1.2ns；而 LUT-5 查表方案将计算压缩为单级查找，延迟降至 0.4ns，降幅可达 67%。这种优化不仅简化了逻辑结构、减轻多级逻辑带来的布线拥塞问题，更从根本上改善了时序特性；在资源效率方面，逻辑查表法同样展现出明显优势，传统 θ 步骤的 5 输入异或树需约 125 个 LUT 实现，而查表优化后仅需 25 个 LUT，资源使用减少 80%。

而 ι 步骤仅作用于单个元素 $A[0][0]$，通过引入轮常数 $RC[ir]$ 执行异或操作以实现状态扰动更新。通过对轮常数特性分析，本团队采用稀疏压缩下的存储优化方法。

观察到，24 组轮常数在 64 个比特位中仅有 7 个特定位为非零值，非零位分别为 $z = \{0, 1, 3, 7, 15, 31, 63\}$，其他 57 位在所有轮常数中始终为 0。基于此特性，对轮常数进行稀疏编码，仅保留有效位的压缩存储形式，减少存储资源。压缩后的轮常数表如表 3.2 所示。

表 3.2　简化后的轮常数表

RC[*ir*]	优化后的常数 (十六进制)	RC[*ir*]	优化后的常数 (十六进制)
0	0x01	12	0x3F
1	0x1A	13	0x4F
2	0x5E	14	0x5D
3	0x70	15	0x53
4	0x1F	16	0x52
5	0x21	17	0x48
6	0x36	18	0x4C
7	0x55	19	0x66
8	0x0E	20	0x79
9	0x0C	21	0x58
10	0x35	22	0x21
11	0x26	23	0x74

在硬件实现中，通过预定义有效比特位索引集合，进行位选取，可直接访问封装字内部的特定位，实现所需的按位异或操作。该方法结合稀疏存储结构，使轮常数的读取与处理过程更加简洁高效。因此，仅需对上述 7 个有效位执行异或逻辑，而对恒零位无需分配任何逻辑电路，从而避免了全宽异或器资源的浪费。该过程的稀疏异或

执行流程如图 3.10 所示。

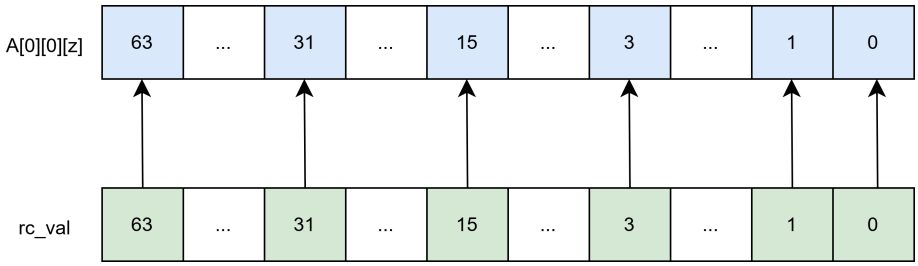


图 3.10　稀疏异或执行流程图

配合循环索引表实现高效实现访问对应位操作，Verilog 实现逻辑如下代码所示，系统为稀疏索引表提前标注了有效操作位位置，使得主循环仅对活跃位执行异或逻辑，其余位维持原值直接传递。

```
//设置系数仅对指定有效位进行运算操作

initial begin
    z_indices[0] = 0; z_indices[1] = 1; z_indices[2] = 3;
    z_indices[3] = 7; z_indices[4] = 15; z_indices[5] = 31; z_indices[6] = 63;
end

always @(posedge clk) begin
    A_out <= A_in; // 默认赋值：先保持A_in原值
    for (i=0; i<7; i=i+1) begin // 对指定z位进行稀疏异或
        A_out[z_indices[i]] <= A_in[z_indices[i]] ^ rc_val[z_indices[i]];
    end
end
```

资源压缩法通过查表优化与稀疏存储的双重技术路径，在保持算法功能完整性的前提下，实现了逻辑资源与时序性能的同步优化，为高并发场景下的能效提升提供了关键技术支撑。

3.3 寄存器复用的采样模块划分

在 Kyber 和 Dilithium 两种格密码算法中，采样操作负责依据哈希函数生成的随机比特流生成多项式系数，不同的采样分布有如表 3.3所示的不同映射规则。

表 3.3　Kyber 与 Dilithium 中的采样

采样名称	功能	分布特征
均匀随机分布	生成矩阵与掩码	取值为 $\mathbb{Z}_q$ 中各元素的概率严格
中心二项分布	生成噪声向量	取值在 $[-\eta, \eta]$，概率与绝对值负相关
SampleInBall 分布	生成签名挑战值	以各 $\tau/2n$ 的概率取值为 ± 1，否则为 0

然而将上述多种采样机制在硬件中独立部署将导致寄存器资源开销大，控制逻辑复杂，不适用于资源受限车联网级的嵌入式设备，因此本作品设计了统一标准，以将三类采样机制均划分为如表 3.4所示的四级处理模块，其中第三级核心映射模块根据规则不同需分别设计，对其余模块则可实现寄存器资源的复用，在此基础上构建寄存器复用的采样模块。在下文中将以均匀随机采样为例对各级模块进行介绍，然后对其余采样方式的核心映射模块进行介绍，最后对寄存器复用的采样模块进行综合介绍。

表 3.4　采样机制的四级处理模块

模块	功能
Module1 缓冲模块	缓存哈希函数输出的伪随机比特流并控制按需读取
Module2 提取模块	暂存每次提取的待采样数据传递给下一个模块
Module3 核心映射模块	负责数据合法性判定并进行映射
Module4 采样结果模块	存储已采样的数据用以构成完整元素

(1) 独立采样的硬件架构

① 均匀随机采样

用于生成服从独立均匀分布的系数的多项式，以 Kyber 中的矩阵生成算法为例，需要生成规模为 $k \times k$ 的矩阵 A，其中每个元素均为商环 $\mathbb{Z}_{3329}[x]/(x^{256}+1)$ 中的多项式，其 $n = 256$ 个合法系数均从伪随机比特流中采样获得。均匀随机采样的四级模块如图 3.11所示。

Module1 缓冲模块：包含以下两部分组件：

(a) 缓冲寄存器：$R_b \in \{0,1\}^{256}$，用于暂存 Keccak 的输出比特流；

(b) 截取寄存器：$R_t \in \{0,1\}^{128}$，负责控制比特流的截取与对齐。

当截取寄存器剩余位宽不足以支撑当前采样需求时，系统自动从缓冲区补充数据并请求 Keccak 生成新的输出块。

Module2 提取模块：$R_e \in \{0,1\}^{32}$，用于暂存每次提取的待采样数据，在 Dilithium 模式下存储 1 组 24 比特数据，生成 1 个系数；$w = 2$ 时，在 Kyber 模式下存储 2 组

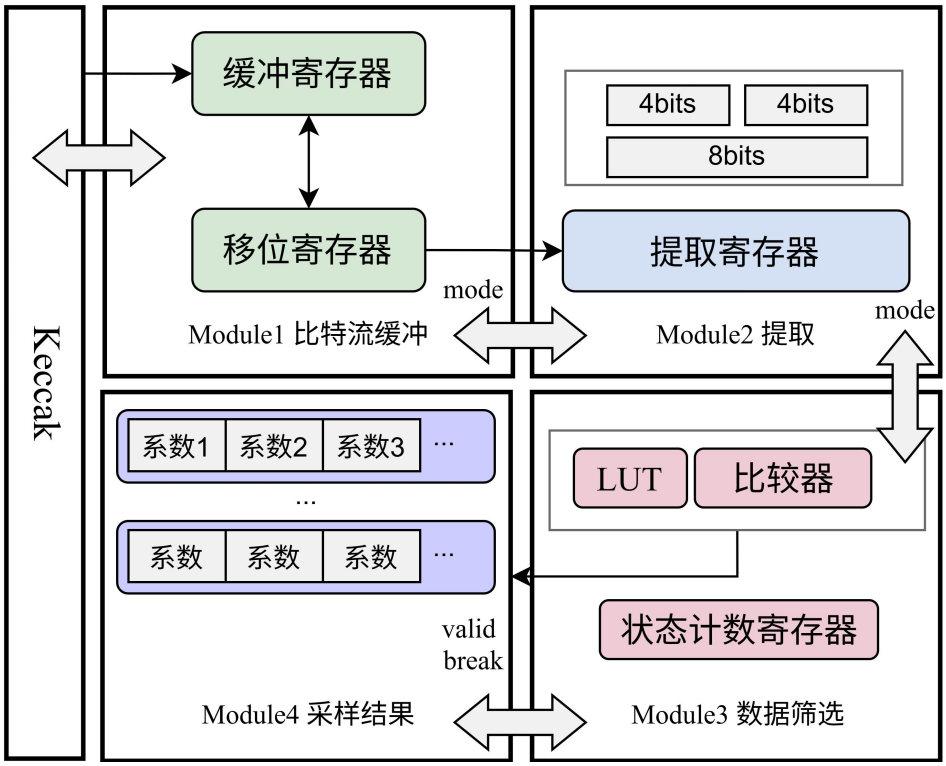


图 3.11　均匀随机采样硬件模块设计

12 比特数据，并行生成 2 个系数。

Module3 核心映射模块：负责数据合法性判定与采样，包含以下两部分功能组件：

(a) 比较与筛选逻辑模块：设采样的比特流为 B，映射函数 $f : B \to Z$ 将比特流转换成十进制整数，合法的采样值集合为 $S = \{x \in B | f(x) \leq q\}$。为提升筛选效率，设计中引入高位 LUT 预筛逻辑，用于提前剔除明显非法样本。

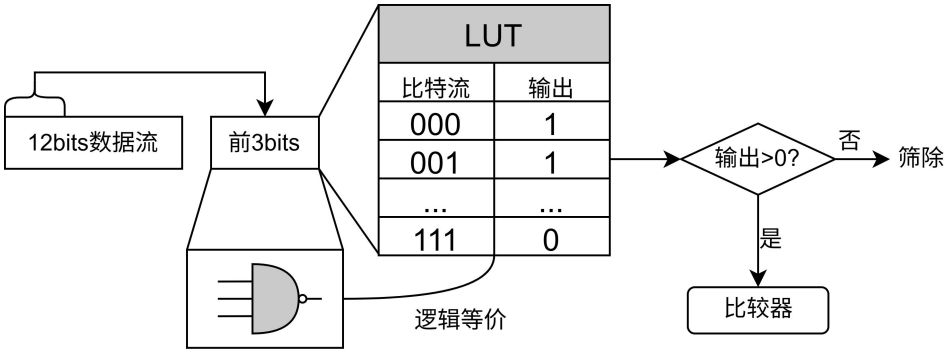


图 3.12　均匀随机采样 Module3 执行逻辑

如图 3.12所示的采样逻辑，实质为提取 B 的前三位 $b = (B_1, B_2, B_3)$，判断

$$b \neq 111 \implies b \in S, b = 111 \implies b \notin S$$

该逻辑可用布尔函数 $\neg(a \wedge b \wedge c)$ 简化实现以减小面积开销。

(b) 状态计数寄存器：用于记录已接受的合法采样系数数量并控制终止。记已采

样的合法系数的有序集合为 S_{acc}，其顺序与采样所得顺序一致，则 $|S_{acc}| = n = 256$ 时触发终止信号，并通知 Module4 输出完整多项式。

在并行提取两个候选系数的情况下，比较与筛选逻辑模块可同步处理 2 个值；写入阶段采用串行顺序写入策略，以保持系数顺序一致性。其调度流程如图 3.13 所示。

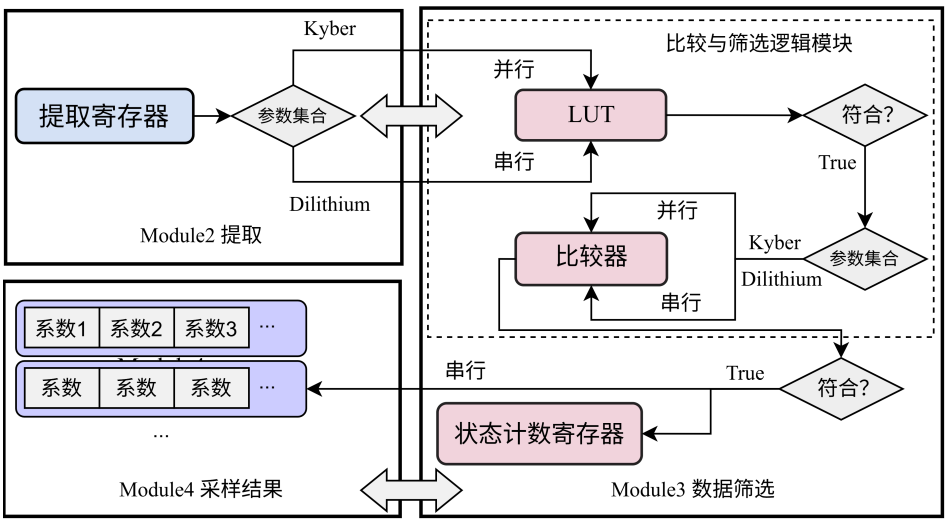


图 3.13　均匀随机采样 Module3 调度逻辑

Module3 向 Module4 传输 2 比特位宽 valid 信号，表示当前轮次生成的合法系数数量和模式：00 表示没有生成合法系数，01 和 10 分别代表在 Kyber 模式下生成了 1 个或 2 个合法系数，11 代表在 Dilithium 模式下生成了 1 个合法系数。采样终止信号 break 由状态计数器生成，当计数达到阈值时自动升为高电平，无需独立信号线传输。

Module4 采样结果模块：负责存储采样生成的多项式系数，由 52 个 128 比特存储的寄存器组成，支持两种存储模式：在 Dilithium 模式下，每个寄存器可存储 5 个 24 比特系数；在 Kyber 模式下，每个寄存器可存储 10 个 12 比特系数，结合并行提取策略，每轮可写入 2 个系数。记有序集合的序列为 $S_{acc} = (c_1, c_2, ..., c_n)$，则多项式向量 $P = (c_1, c_2, ..., c_n)$。

② 中心二项分布采样

用以实现 Dilithium 中噪声向量生成，要生成规模为 $l \times 1$ 的向量 $\mathbf{s}_1$，其每一个元素都是具有 n 个系数的多项式，其系数服从参数为 η 的中心二项分布。采样时，对于向量中每个元素均利用随机数种子生成待采样比特流。

中心二项分布采样的硬件结构可以分为四个模块, 其中 Module1,Module2 与 Module4 仅在规模上与均匀随机采样不同，在 Module3 的执行上则存在显著差异。

Module3 核心映射模块：执行采样的映射值判定与采样状态控制，内部包含如下两部分功能组件：

(a) 汉明重量计算模块：设采样的比特流为 B，划分为两个等长子序列 (B_1，B_2)，定义映射函数 $f : B \to Z$ 和汉明重量算子 HW，则 $f(B) = HW(B_1) - HW(B_2)$ 为采样值，并输入至 Module4 进行存储。

(b) 状态计数寄存器：用于记录已接受的合法采样系数数量并控制终止。记已采样的合法系数的有序集合为 S_{acc}，其顺序与采样所得顺序一致，则 $|S_{acc}| = n = 256$ 时触发终止信号，并通知 Module4 输出完整多项式。在参数 $\eta = 2$ 时，LUT 查表与差值计算可并行进行；写入阶段仍采用串行顺序写入方式，以确保系数在存储中的位置一致性。其调度逻辑如图 3.14所示。

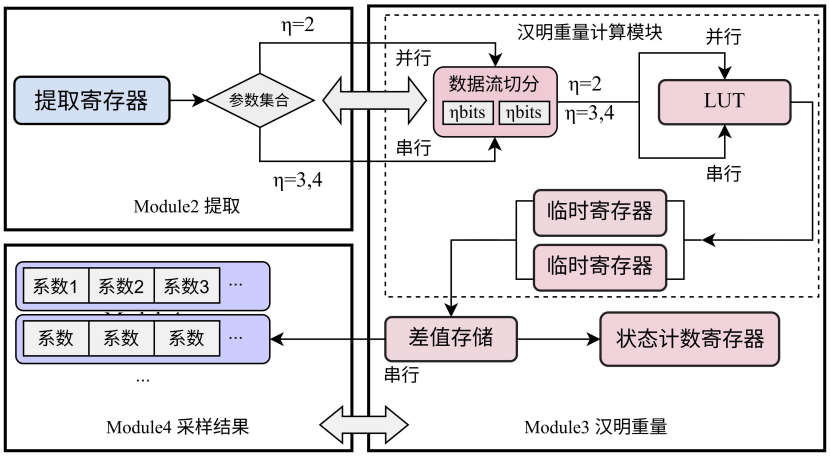


图 3.14　中心二项分布采样 Module3 调度逻辑

③ SampleInBall 采样

用于 Dilithium 中稀疏多项式的生成，多项式 c 具有 n 个系数，其中恰有 τ 个系数非零。采样时，对于向量中每个元素，利用前置数据生成待采样比特流。

SampleInBall 采样的硬件结构可划分为四个模块，其中 Module1,Module2 与 Module4 仅在规模上与均匀随机采样不同，在 Module3 的执行上则存在显著差异。

Module3 核心映射模块：负责判断位置索引的合法性并控制采样状态更新，内部包含如下两个功能组件：

(a) 去重判断模块：定义索引空间 I 为多项式中所有可能系数的集合，其大小为 n。系统维护一个状态向量 S，用以记录每个位置是否已被选中；初始时 S 的所有元素均为 0。每次提取一个位置索引 i，系统首先检查 $S[i]$ 的取值：若为 0，表示该位置尚未被选取，则当前索引被接受并将 $S[i]$ 更新为 1；若为 1，则该位置已被占用，当前样本被判定为无效并丢弃，系统继续提取新的比特流进行下一次尝试。

对于每个被接受的位置索引，系统再读取随后的一个符号比特，并通过查表映射确定系数的符号值。映射规则为：若符号比特为 0，则对应系数记为 −1；若符号比特

为 1，则对应系数记为 +1。

（b）状态计数寄存器：用于记录已接受的合法采样系数数量并控制终止。记已采样的非 0 系数的有序集合为 S_{acc}，其顺序与采样所得顺序一致，则 $|S_{acc}| = \tau$ 时触发终止信号，并通知 Module4 输出完整多项式。其调度逻辑如图 3.15所示。

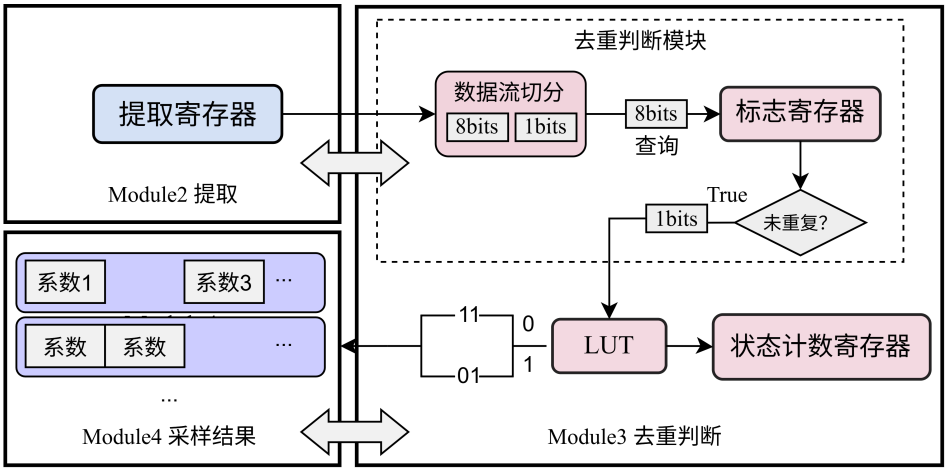


图 3.15　SampleInBall 采样 Module3 调度逻辑

综上所述，三类采样算法在硬件实现中具备统一的结构特征，均依赖随机数种子与哈希函数生成伪随机比特流，且均由比特流缓存、数据提取、映射与结果写入四级模块组成，在此基础上，本作品提出统一架构设计并展开优化。

(2) 寄存器复用的统一采样模块

统一采样模块针对三类采样方式均具备如表 3.4所示四级处理模块的结构共性，设立了采样执行的标准框架，此时 Module3 用于分支执行特定采样算法下的映射与判断逻辑；其余模块则执行相同功能，所需寄存器资源规模取各采样方式中最大值，以实现硬件资源复用。统一采样系统的四级处理模块结构如图 3.16所示。

Module1 缓冲模块：包含以下两部分寄存单元。

(a) 缓冲寄存器：$R_b \in \{0,1\}^{256}$，用于暂存 Keccak 的输出比特流；

(b) 截取寄存器：$R_t \in \{0,1\}^{128}$，负责控制比特流的截取与对齐。

此时由 mode 信号选取所需采样模式以判定采样位宽。

Module2 提取寄存器模块： $R_e \in \{0,1\}^{32}$，用于暂存每次提取的待采样数据。在均匀随机采样模式下，Kyber 模式可并行处理 2 个样本；在 CBD 采样模式下，η 为 2 时每次并行处理 6 个样本，η 为 3 时并行处理 4 个样本，η 为 4 时并行处理 3 个样本；在 SampleInBall 采样模式下，每次并行处理 2 个样本。

Module2 向 Module3 输入 mode 信号以明确映射模式

Module3 核心映射模块：负责对候选采样数据进行合法性判定、系数映射与采样

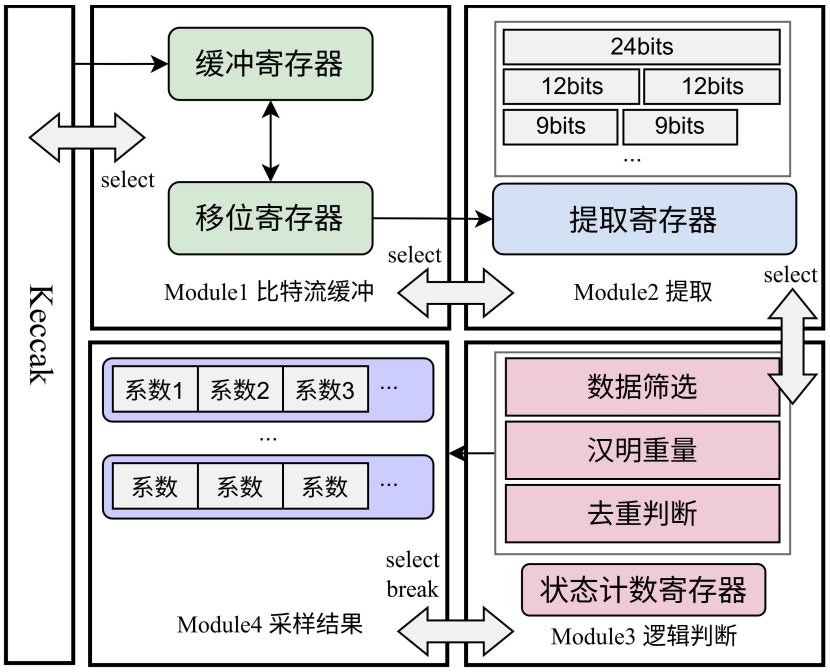


图 3.16　协同采样硬件模块设计

状态控制，内部包含两个子模块组件。

（a）逻辑判断模块：内部划分为可独立调度的三个子模块以供不同的采样方式调用。在均匀随机采样模式下，采用 LUT 查表预筛 + 比较器的比较与筛选逻辑模块；在 CBD 采样模式下，采用 LUT 查表映射 + 临时存储 + 差值存储的汉明重量计算模块；在 SampleInBall 采样模式下，采用 LUT 查表映射 + 标志寄存器的去重判断模块，均支持并行操作。

（b）状态计数寄存器：统一状态计数器用于记录合法样本的数量，从而控制采样终止条件。均匀随机分布采样与中心二项分布采样的终止阈值统一设为 256，SampleInBall 采样的终止阈值则为 τ。

Module3 向 Module4 传输 4 比特宽的 break 控制信号与 3 比特宽的 select 信号。其中，break 信号的高 1 位用于标志采样是否完成，低 3 位用于编码当前轮次生成的合法系数数量。select 信号用于指示 Module4 选择相应的存储模式与写入方式。

Module4 采样结果寄存器：由最多 52 个 128 比特寄存器组成，用于存储采样生成的多项式系数。接受 select 控制信号并选择存储逻辑。在均匀随机采样与中心二项分布采样模式下，系数按顺序写入；在 SampleInBall 采样模式下，则依据位置索引进行稀疏写入。

寄存器复用的采样模块设计在保持完整功能支持的同时，显著提升了硬件资源利用率与控制效率：通过共享寄存器资源，能够在节省 2321 比特的空间（约占总空间

的 25%）的同时提升了系数写入的并行程度；此外，系统仅需 2 种控制信号即可实现模块间调度与采样状态管理，极大简化了逻辑控制和接口配置。

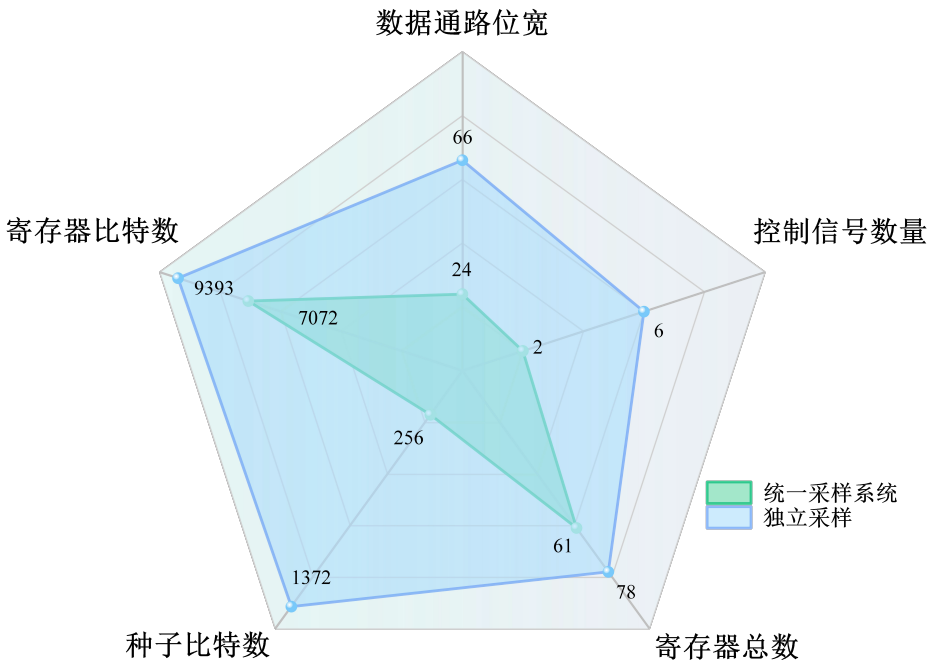


图 3.17　统一采样系统与独立采样的数据对比

图 3.17展示了统一采样系统与独立采样主要数据的对比。统一采样系统结合多级种子管理策略的协同设计，有效降低了寄存器资源开销与信号调度复杂度，在保障功能完整性的同时实现结构集成度与执行效率的双重优化。

3.4 D-KPolyArch 存储架构

在 Kyber 与 Dilithium 两类后量子密码算法的硬件加速实现中，多项式运算模块因其高频繁度的乘法、加法与模约简计算，承担了主要的计算负载与硬件资源消耗，已成为系统吞吐性能与资源利用效率的关键瓶颈。

虽然 Kyber 与 Dilithium 在多项式计算框架上具有高度一致性，均以 NTT 变换实现，且以多项式乘法为核心运算，但二者在模数大小、系数位宽、NTT 结构、乘法模式与模约简路径等具体实现细节上存在显著差异，若直接复用通用硬件架构将不可避免地导致控制复杂性提升、逻辑资源冗余以及性能受限等问题。

针对上述异构性问题，本章聚焦于多项式运算模块的统一优化设计，提出统一多项式运算架构（D-KPolyArch）如图 3.18所示。该架构以提升跨算法的资源复用率与系统吞吐能力为目标，围绕以下两个核心方向进行优化设计：

（1）D-K 存储架构设计：针对两类算法在多项式规模、系数位宽与数据流特性上的差异，提出统一位宽 24 bit 的交织式 4-Bank 存储架构，结合就地 NTT 与混合进制

地址映射算法，实现存储资源的高效复用与数据并行访问能力提升；

（2）双 ALU 解耦设计：面向两类算法在乘法路径与模约简逻辑上的显著差异，设计双通路解耦架构，分别优化运算单元内部数据流通路，有效避免路径冲突与共享控制逻辑导致的控制复杂度膨胀。

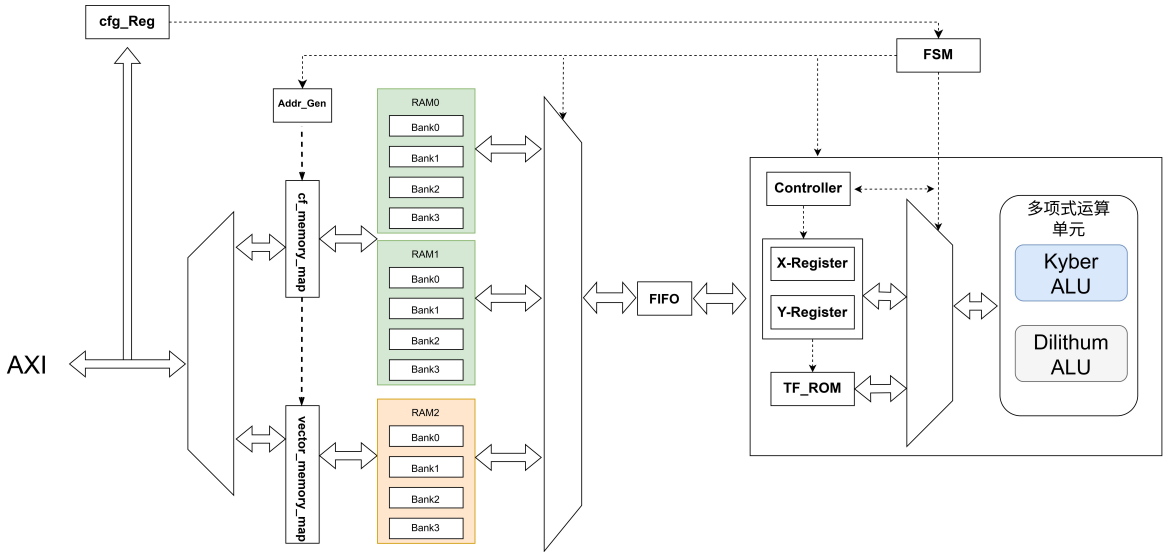


图 3.18　D-KPolyArch 架构设计

3.4.1 D-K 存储架构设计

图 3.19展示本作品设计的 D-K 存储架构，由多项式 RAM、地址生成单元、内存映射单元、控制单元和 FIFO 缓冲区组成，实现了对两种算法不同存储模式的高效支持，在保证零冲突的同时显著提升存储资源复用率。

每个多项式 RAM 由 4 个 Bank 组成，每个 Bank 由一个或多个双端口 BRAM 构成，一个端口用于读取，另一个端口用于写入，实现了算法的全流水线处理。单个多项式的 256 个系数通过内存映射方法均匀地分布在这 4 个 Bank 中，即每个 Bank 负责存储 64 个系数。同时，采用就地 NTT，每个周期运算结束后，所得中间结果将直接覆盖到原始逻辑地址对应的存储单元中。经过完整的 NTT 循环过程后，多项式 RAM 中的相应多项式会完全转换到 NTT 域中。

地址生成单元根据当前 NTT 阶段和并行蝶形单元（BFU）的数量等参数，生成每个周期所取系数的逻辑地址；内存映射单元将逻辑地址映射为实际地址。本作品采用的 NTT 单元的基本组成是基-2 蝶形单元，每周期读取 4 个系数。

FIFO 缓冲区作为系数读取与计算单元之间的数据流调节模块，用于对每轮从 4-bank 交织存储中并行读取的 4 个多项式系数进行缓存，并按需分批向后端计算单元输出；系统每次从 FIFO 中提取两个系数，送入流水线 BFU 模块进行多项式运算。

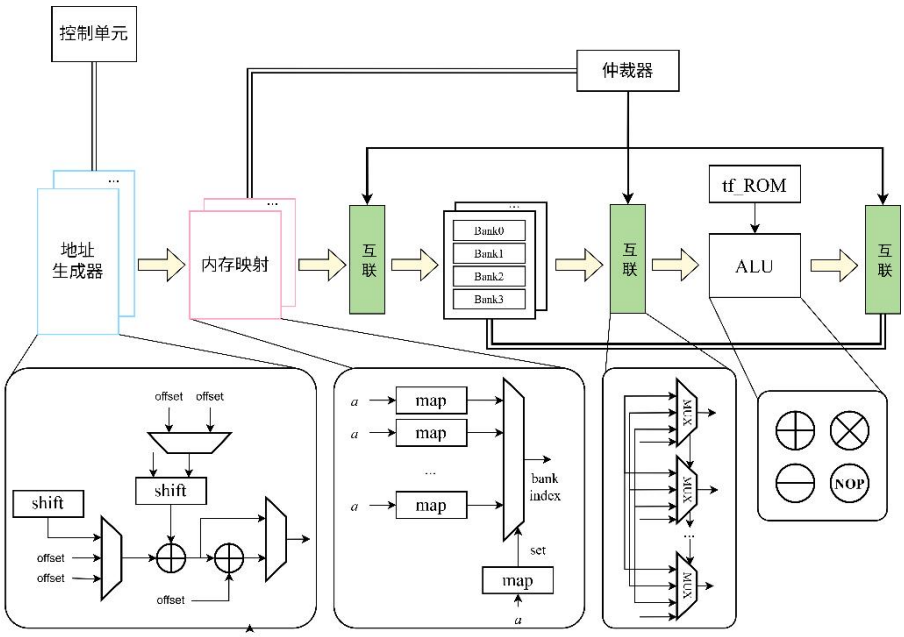


图 3.19　D-K 存储架构设计

顶层控制逻辑包括控制单元和仲裁器。控制单元负责多项式运算处理流程的调度控制，如启动、阶段切换、同步以及指令生成；根据多项式乘法的不同状态，如初始状态、NTT 状态、点乘状态、INTT 状态等，给出不同的地址状态信息和读写使能信号。仲裁器负责在实际运行过程中对可能的并发请求进行实际调度，决定优先访问顺序，保证流水线不发生数字冒险，确保硬件系统的健壮性。

(1) 统一位宽的交织式 4-Bank 存储结构

针对 Dilithium($q = 23$) 和 Kyber($q = 12$) 在存储需求上的差异，在 4-Bank 交织存储结构的基础上，统一将存储字宽设定为 24 位，实现对两种算法的高效兼容。针对 Dilithium，每个存储字存放一个 23 位系数，高位补零，每拍并行读取 4 个系数；针对 Kyber，每个字的高 12 位存储 a_{2i+1}，低 12 位存储 a_{2i}，$i \in [0, 127]$，每拍读取 8 个系数，并通过专用路由逻辑将奇偶系数自动分配至对应蝶形单元。由于每对奇偶系数共用同一旋转因子，该设计使得旋转因子的读取需求减少 50%，进一步降低了 ROM 访问频率与控制复杂度。

为提升多项式系数在运算过程中的并行读写效率，采用交织式内存管理策略，将输入多项式的数据均匀分布到多个独立的内存 Bank 中，避免传统顺序存储方式下的访存冲突。系统支持每周期并行读取 4 个系数，显著提高了访存带宽与数据并行度。

传统的 NTT 硬件设计通常采用乒乓式的异地存储方式 [17]，即使用两块同等大小的多项式 RAM 交替读写中间结果。虽然该方式便于流水线调度，但显著增加存储资

源的面积开销。为减少这种存储冗余，系统采用就地 NTT 计算方式，在原始数据存储区域直接进行中间结果回写，从而在不增加存储体数量的前提下完成完整的 NTT 转换过程，有效降低了资源占用。

(2) 混合进制地址映射的无冲突访问

4-Bank 交织结构与就地 NTT 面临的核心挑战在于**空间冲突**——每周期同时访问四个逻辑地址，若多个地址被映射到同一物理 Bank，超出 BRAM 双端口的访问能力上限，引发读写竞争和访存失败的情况。尤其在 $stride$ 变化阶段，冲突不可避免，必须通过算法优化解决。现有的包括地址重映射 [17]、向量无冲突调度 [18] 在内等方案，均存在逻辑资源开销大、互联网络复杂等问题。

针对这一难题，本作品创新性提出硬件-算法协同优化的两级解决方案：

①**计算感知地址生成算法**：基于 NTT 计算的蝶形运算规律，动态生成具有时空局部性的访问序列，确保每周期四地址覆盖完整的蝶形操作对。具体如算法3.2展示三层循环控制下生成具有可映射性的逻辑地址序列的算法流程。

②**混合进制地址映射算法**：提出高位-低位分离的混合进制重排机制，通过轻量级位操作，仅需 3 级逻辑延迟即可实现无冲突各 Bank 分布。在硬件层面配合 FIFO 缓冲机制，为计算模块提供稳定的数据供给，彻底消除流水线阻塞问题。具体如算法3.3所示。

Algorithm 3.2 计算感知地址生成算法

Require: $N = 256$（多项式系数个数）, $d = 2$（蝶形对个数）, $R = 2$（基数）

for $p \leftarrow 0$ to $\log_2(N) - 1$ **do**
  $J \leftarrow 2^p$
  $RJ \leftarrow 2 \times J$
  for $k \leftarrow 0$ to $\frac{N}{RJ} - 1$ **do**
    for $i \leftarrow 0$ to $d - 1$ **do**
      $addr1 \leftarrow k \times RJ + i \times d$
      $addr2 \leftarrow addr1 + J$
      输出 $(addr1, addr2)$

结合基-2 蝶形结构的访问模式，设计了计算感知的多层地址调度生成器，算法3.2通过三层循环参数（p 层次，k 批次，i 蝶形单元数）实现每一拍输出两对系数。首先根据层次 p 计算当前阶段的蝶形跨度 J 及批次数量 RJ，随后在每一批次中顺序生成地址对。内层循环通过步进变量 i 在每个批次内逐个计算两个相邻的蝶形输

入地址，作为一对输出。

为实现系数在 4-Bank 中的均匀无冲突分布，进一步设计了 NTT 蝶形逻辑地址的混合进制映射机制如算法3.3 所示，将逻辑地址的高位和低位分别映射为 Bank 编号与组内偏移部分，仅依赖移位与模运算即可高效完成 Bank 分配与组地址计算。

Algorithm 3.3 NTT 混合进制地址映射

Require: a, N, d, B；其中 a 为旧地址，N 表示多项式系数个数，d 为蝶形单元对数，B 为 Bank 数目

Ensure: BI, BA；其中 BI 表示存储组编号，BA 表示组内偏移

1: 计算旧地址总位宽 $T = \log_2 N$，组编号位宽 $M = \log_2 B$，高位宽度 $C = T - M$

2: 将 a 表示为二进制 $(a_{T-1}, \ldots, a_0)$

3: 低 M 位转为 B 表示组内偏移 $\quad (b = a_{M-1}, \ldots, a_0)_B$

4: 高 C 位转为 M 组 $\quad (m = \{a_{T-1}, \ldots, a_M\})_R$

5: 混合进制编码：$a = [m_1, m_2, \ldots, m_B]$

6: 定义 $SN = (\sum m_i) \bmod R$

7: 计算组编号：$BI = (b + SN \cdot d) \bmod B$

8: 计算组内偏移：$BA = a \gg M$

9: **return** BI, BA

通过对算法3.2生成的逻辑地址进行混合进制重排，仅需移位、模加和位拼接三级轻量级运算即可完成 Bank 编号与组内偏移的快速解析。在确保彻底消除空间冲突的同时，运算单元可完全映射为硬件友好型逻辑电路，具备较低的资源开销与较高的执行效率。

(3) 基于 3-RAM 的动态矩阵生成与调度架构

前述两节设计了对单个多项式 NTT 运算的高效存储，构建了具备并行访问与冲突规避能力的 4-Bank 结构及地址映射机制。然而在 Kyber 和 Dilithium 算法中，基于 NTT 运算的大规模矩阵-向量乘法作为高频核心操作，若采用传统静态存储方式，矩阵 A 的完整存储开销将高达数 86KB，且矩阵元素在乘法计算过程中仅被访问一次，资源复用性极低，严重制约硬件可扩展性。为解决该问题，本作品提出基于 3-RAM 的动态矩阵生成与调度架构仅在运算过程中按需动态生成矩阵 A 的各行多项式，并结合三块 RAM 的功能划分，实现“读取-计算-回写”流水线，继承并扩展了 4-Bank 存储架构和混合进制内存映射的无冲突访问优势，保障跨周期调度下的数据流高效连续，使得 D-K 存储架构的存储开销降低 87.5%。

下述以 *Dilithium* 算法中 $t = \tilde{A} \cdot \tilde{s}_1 + s_2$ 为例，说明基于 3-RAM 的动态矩阵生成与调度架构的运作流程，该步骤在硬件实现中矩阵-向量乘法的计算过程如(3.1)所示。

$$t_i = \text{INTT}\left(\sum_{j=0}^{l-1} \tilde{A}_{ij} \cdot \tilde{s}_{1,j}\right) + s_{2,i}, \quad i \in [0, k-1] \tag{3.1}$$

其中 $\tilde{A}_{ij}$ 表示矩阵 A 在 NTT 域中的第 i 行第 j 个多项式元素，$\tilde{s}_{1,j}$ 表示向量 s_1 在 NTT 域中的第 j 项，$s_{2,i}$ 表示向量 s_2 中的第 i 项。

表 3.5　3-RAM 结构

操作类型	数据流	各 RAM 存储的数据
NTT/INTT	$A(x) \to \text{NTT}(A)$	单独使用，读写切换
多项式乘法	$\tilde{A}_{ij} \cdot \tilde{s}_{1,j}$	RAM0 存 $\tilde{A}_{ij}$，RAM1 存 $\tilde{s}_{1,j}$，结果写回 RAM0
累加	$t_{i+1} = t_i + \tilde{A}_{ij} \cdot \tilde{s}_{1,j}$	RAM2 读 t_i，结果写回 RAM2

3-RAM 即 RAM0、RAM1 与 RAM2 三组独立的交织式 4-Bank 存储单元，其中 RAM2 用于存储多项式向量运算的累加和 t_i，RAM0 和 RAM1 分别存储点乘操作中的两个多项式 $\tilde{A}_{ij}$ 和 $\tilde{s}_{1,j}$，进行的操作类型和数据流如表 3.5 所示。

对于就地 NTT 进行扩展，所有中间结果（包括点乘、累加）均直接写回原存储体，图 3.20展示了具体操作下的数据流路径。

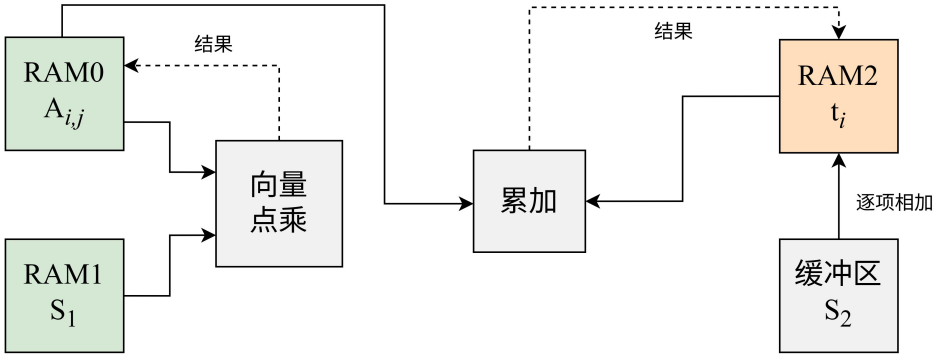


图 3.20　多项式矩阵-向量乘加过程的数据流路径

Step1: 加载输入数据。RAM1 存储 $\tilde{s}_1$ 向量中的所有多项式；RAM2 在计算每个 t_i 前清零，s_2 通过 FIFO 动态流入。

Step2: 动态生成矩阵 A_{ij} 第 i 行的全部多项式，并依次写入 RAM0。

Step3: 完成第 i 行对应的矩阵-向量乘法累加。对列索引 $j \in [0, l-1]$，首先从 RAM0 和 RAM1 中分别取出 A_{ij} 和 $\tilde{s}_{1,j}$，并进行相乘，结果写回 RAM0；再将最终 RAM0 的结果与 RAM2 中数据相加，结果写回到 RAM2。

Step4: 对于 RAM2 的结果进行 INTT 操作。

Step5: 从缓存区取出 $s_{2,i}$，与 RAM2 逐项相加。

基于 3-RAM 的动态矩阵生成与调度架构把多项式矩阵 A 的存储复杂度从 $O(kln)$ 降至 $O(ln)$。同时 s_2 依靠小型缓冲区按需逐项供给，实现与核心计算模块间的速率匹配。动态调度有效规避了传统预存方式造成的冗余存储与带宽浪费，显著提升了存储资源利用率，为资源受限芯片平台提供了兼具灵活性与高效性的解决方案。

本作品实现的 D-K 统一存储架构旨在解决算法在多项式存储结构上的位宽不统一、访存冲突频繁与矩阵运算存储负载过重等核心矛盾：①采用 24bit 统一位宽的 4-Bank 交织存储结构，支持两种算法的系数存储模式，通过共享旋转因子机制降低 50% 的 ROM 访问开销；②提出混合进制地址映射算法，结合计算感知地址调度，实现零冲突访存访问，减少逻辑资源约 60%；③构建 3-RAM 动态调度架构，实现矩阵按需生成与流水化累加处理，整体存储开销压缩 87.5%。该架构保障通用性与高效性的同时，提升后量子密码算法在嵌入式平台上的适配性与性能表现。

3.4.2 双 ALU 解耦设计

D-K 存储架构的设计为两类算法在数据规模与访存方式上的兼容提供了稳定高效的数据流通道。针对 Kyber 与 Dilithium 在乘法路径、模约简方式与位宽需求上的差异，本节进一步提出双 ALU 解耦设计。通过物理隔离构建适配两类算法的独立 ALU 通路，并通过内部逻辑单元复用机制，实现计算性能和控制复杂度的协同优化。

对于两算法支持的不同乘法操作与模约简类型，本作品实现独立的两套 ALU：Kyber-ALU 结合交错乘法 CWM 与 Montgomery/Barrett 约简逻辑，设计专用奇偶位交叉计算结构与懒惰模约简机制，减少乘法器调用次数与临界路径长度；Dilithium-ALU 依据高位宽乘加操作（MAD）与 Plantard 模约简需求，构建融合型 MUL+REDC 模块，配套定长五级流水线架构，优化高位宽数据搬移压力与流水节奏稳定性。

(1)Kyber 的 ALU 设计

针对本算法中 Montgomery 域转换、NTT/INTT 变换和 CWM 交错乘法三类运算路径的差异，本节对其硬件实现流程逐一进行深度解构。提取多条路径间共用的乘法器、模约简器与加法器等模块，设计一套专用 ALU 架构，采用统一算术核心与可配置控制路径，支持运算资源复用。FSM 控制器根据配置寄存器字段动态启用对应运算路径，降低控制逻辑复杂度。

① Montgomery 域转换

为提升模乘效率，Kyber 中所有乘法操作需在 Montgomery 域内完成。因此，在进

入 NTT 变换前，先将输入系数与旋转因子转换至 Montgomery 域，可以为后续乘法与约简模块减少模乘开销。如图 3.21所示，输入值与 R^2 进行相乘，结果送入 Montgomery 模约简模块。该条路径使用到 1 个乘法器、1 个 Montgomery 模约简模块。

图 3.21　Montgomery 域转换路径

② NTT/INTT

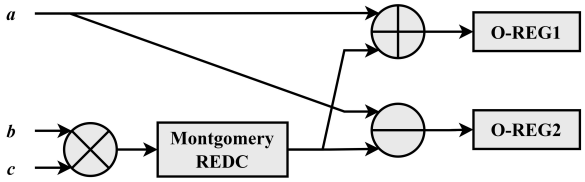


图 3.22　NTT 操作路径

NTT 依赖蝶形单元 CT 结构，操作路径如图 3.22所示，INTT 依赖蝶形单元 GS 结构，操作路径如图 3.23所示。两条路径均使用到 1 个乘法器、1 个 Montgomery 模约简模块、1 个加法器、1 个减法器。

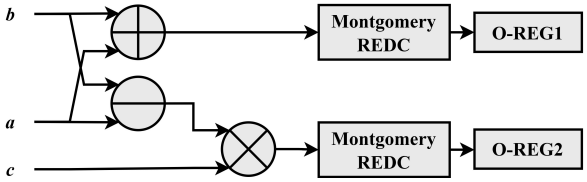


图 3.23　INTT 操作路径

③ CWM 操作

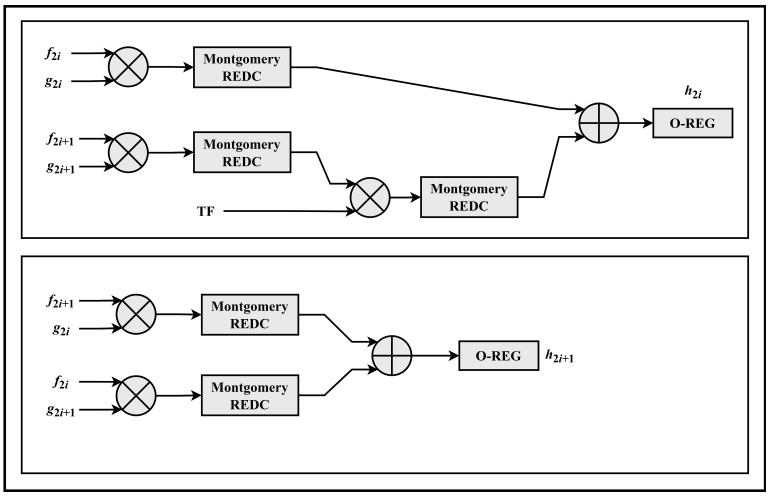


图 3.24　CWM 操作路径

多项式乘法采用 CWM 交错策略 [19]，将奇偶数系数分开处理，构建图 3.24所示操

作路径。路径共使用到 3 个乘法器、3 个 Montgomery 模约简模块、1 个加法器。

为进一步配合 CWM 操作的分阶段计算特性，ALU 中构建了如图 3.25所示的轻量级交错流水线结构，采用“计算—等待—输出合并”的循环机制，实现每个时钟周期输出一个新系数。根据图 3.24可知 h_{2i} 的生成要比 h_{2i+1} 的生成多两个时钟周期，于是设计 h_{2i} 的计算早于 h_{2i+1} 一个时钟周期开始，其结果晚于 h_{2i+1} 一个周期输出的并行流水线设计，实现每拍输出一个结果，保证持续满吞吐输出。

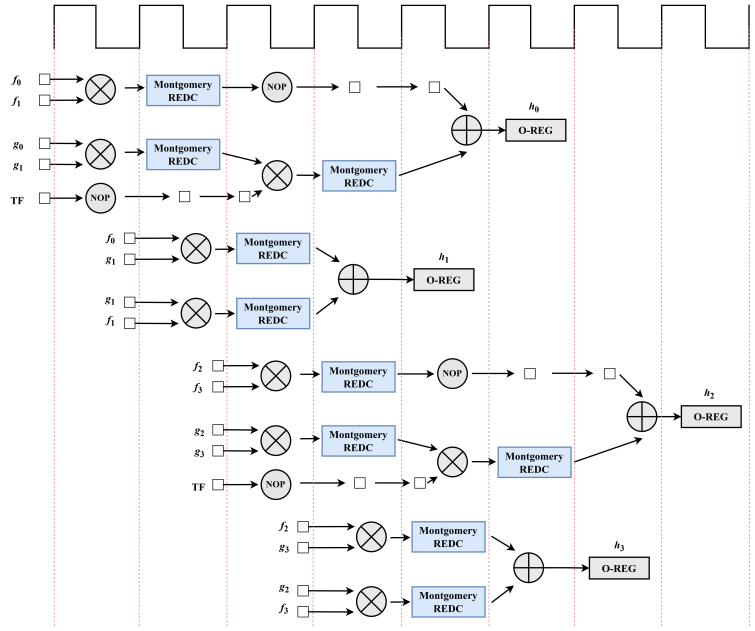


图 3.25　Kyber 的 CWM 流水线设计

由上述三个操作路径图可知，在底层乘法、加法与模约简等逻辑中存在高度结构共性。因而本作品提取共用基础算子模块，设计如图 3.26所示的 ALU，实现所需多项式运算操作，有效压缩了乘法器与模约简器等高开销算子的实例数。

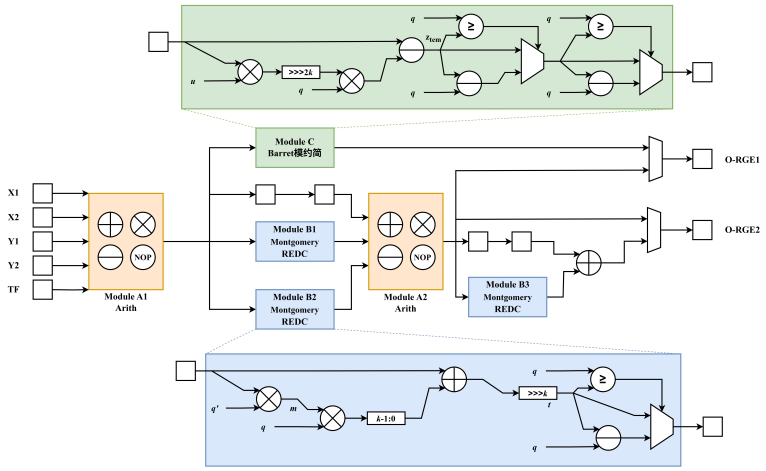


图 3.26　Kyber 的 ALU 设计

（2）Dilithium 的 ALU 设计

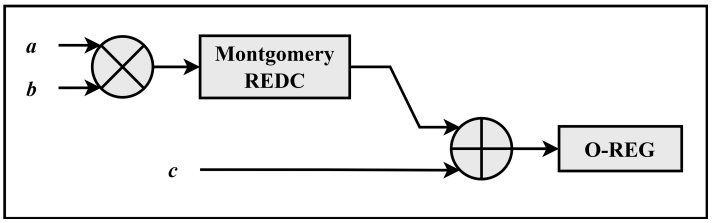


图 3.27　MAD 操作路径

相较于 Kyber 算法，Dilithium 在多项式计算路径上呈现出更高的结构复杂度与数据宽度挑战，主要体现在高位宽乘法（23 bit）导致中间数据膨胀。为缓解该问题，设计专用 Dilithium-ALU，构建乘法器与 Montgomery 模约简（MUL+REDC）融合计算单元，并设计统一 5 级定长流水线，以实现高性能、低延迟的硬件加速，同时优化了资源利用率和控制复杂度。

Montgomery 域转换与 NTT/INTT 操作的路径与 Kyber 相同，但 Dilithium 算法频繁使用包含挑战值的多项式乘加（MAD）操作，路径如图 3.27所示。乘法系数 a, b 相乘后通过 Montgomery 模约简模块，结果与待累加系数 c 进入加法模块完成累加操作。该路径使用到 1 个乘法器、1 个 Montgomery 模约简模块、1 个加法器。

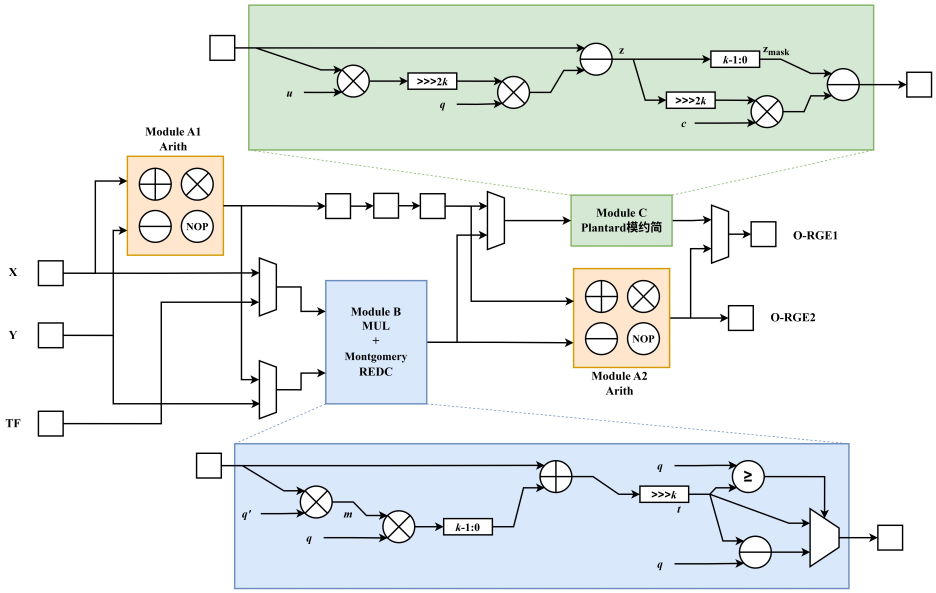


图 3.28　Dilithium 的 ALU 设计

由于 Dilithium 算法的位宽为 23bit，在对系数进行 Montgomert 域转换时，中间乘积数据宽度高达到 87bit，显著增加中间寄存器的宽度与数量，并引入额外的数据搬移和控制信号开销，影响整体计算时序和资源效率。因此，设计融合的**乘法-模约简（MUL+REDC）**计算单元，模块内部自带中间寄存器和锁存，仅需要一个启动信号即

可完成两个步骤，显著减少了数据通路的延迟。通过共用底层逻辑，设计如图 3.28的 Dilithium-ALU，在硬件资源消耗、控制复杂度与时钟性能之间实现优化平衡。

为平衡性能与复杂度，采用统一的 5 级定长流水线结构。如图 3.29所示，所有操作均按“读取-乘法-Montgomery 约简-加法-写回”执行，确保了不同运算路径的时序一致性，避免了动态调度的复杂性，同时实现每周期输出一个结果，达到**满吞吐性能**。

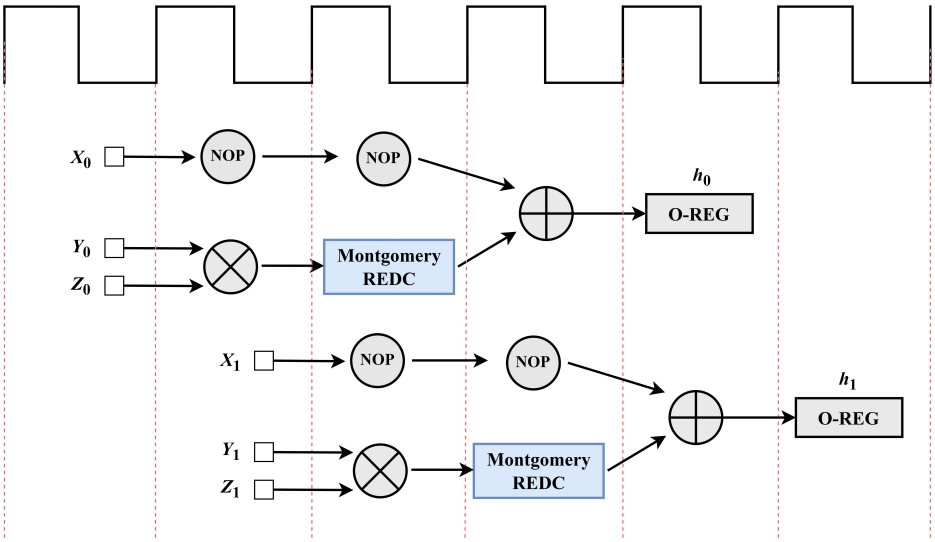


图 3.29　统一 5 级流水线设计

针对两类算法在运算结构上的异构性，本节提出双 ALU 解耦设计，构建 Kyber-ALU 与 Dilithium-ALU，针对不同路径的通用算术单元进行融合复用，使关键模块平均复用率提升约 30%。通过统一定长流水线设计，显著降低控制复杂度，实现双算法兼容下的资源共享与性能协同，为 D-KPolyArch 架构提供高效稳定的算术支撑。

第四章 作品测试与分析

4.1 基于规划求解的算子结构最优构型

为在资源受限的车联网级平台上实现后量子算法的高效部署，本作品基于规划求解实现了算子结构最优化——针对系统内主要算子（Keccak、NTT 与采样模块等）在计算效率与面积占用之间的权衡关系，建立多种可选硬件实现方案，并将各方案的综合结果通过综合工具与 RTL 仿真进行量化建模。在此基础上，以车载级 FPGA 资源上限为约束，引入“总面积最小化”条件，构建以系统计算效率最大化为优化目标的算子级规划求解模型。该模型确定了在限定规模内的最优配置方案，从而实现系统性能的全局最优化。

设算子集合 $\mathcal{O}$ 与算法集合 $\mathcal{L}$ 为

$$\mathcal{O} = \{\text{Keccak}, \text{NTT}, \text{Sampling}\}, \mathcal{L} = \{\text{Kyber}, \text{Dilithium}\}.$$

定义主要参数如表 4.1所示：

表 4.1　规划求解模型中的变量与参数定义

符号	含义
a_o	算子 o 的单实例面积（LUT 数量）
τ_o	算子 o 的单实例处理速率，由仿真与综合结果换算获得
$d_{o,\ell}$	算法 ℓ 在一次操作中对算子 o 的调用次数
$A_{\max}$	系统总面积约束
r_o	算子 o 的实例数量（整数变量）
z_ℓ	算法 ℓ 的上层吞吐量（ops/s）
t	全局最小吞吐量变量，用于定义最优化目标

为保证系统的均衡性能，规划目标设定为最大化算法吞吐率下界，以保证 Kyber 与 Dilithium 算法在资源受限条件下均可获得最优且均衡的执行效率，即：

$$\max\ t \quad \text{s.t.} \quad z_\ell \geq t, \quad \forall \ell \in \mathcal{L}. \tag{4.1}$$

此外在规划求解时引入如下约束条件：

面积约束：用于限制算子实例化后的总逻辑资源不超过平台可用 LUT 容量。

$$\sum_{o \in \mathcal{O}} a_o \cdot r_o \leq A_{\max}. \tag{4.2}$$

算子处理速率约束：表示各算子实例的计算效能必须支撑上层算法的计算需求。

$$\tau_o \cdot r_o \geq \sum_{\ell \in \mathcal{L}} d_{o,\ell} \cdot z_\ell, \quad \forall o \in \mathcal{O}. \tag{4.3}$$

变量非负约束：确保求解具有现实意义

$$r_o \in \mathbb{Z}_{\geq 0}, \quad z_\ell \geq 0. \tag{4.4}$$

通过求解上述整数线性规划模型，系统自动确定在有限逻辑资源下的算子实例数量与配置比例，实现系统整体吞吐率的最优分配。

模型求解过程中所用的算子参数均来源于 RTL 综合与 Vivado 仿真结果：

表 4.2　模型求解使用的算子参数

算子名称	主要参数
Keccak	225 MHz 下吞吐率 6.71 Gbps，面积约 2500 LUT
NTT	时延 22.86 μs，面积约 1926 LUT
Sampling	面积约 600 LUT，处理速率 1.2×10^6 calls/s

当面积上限设定为 $A_{\max} = 5000$ LUT 时，优化结果表明系统优先增加 NTT 实例数量可有效解除瓶颈；当面积进一步放宽至 $A_{\max} = 6000$ LUT 时，规划模型不再单纯地扩大单一算子的数量，而是形成更均衡的资源分配：NTT 实例数增加 1 个、Keccak 保留 1 个、Sampling 维持 1 个。此时系统整体吞吐率较 5000 LUT 方案再提升约 35% 的同时，Kyber 与 Dilithium 的最小吞吐率差距缩小至 5% 以内。

这一结果表明，在资源充裕区间，规划求解模型能够主动进行算子层级的结构平衡，不仅提升性能上限，还实现了多算法间的动态协同，避免系统性能畸形。

随后，本节依据上述求解结果，对各模块及整体系统的性能进行了仿真推演与对比验证。所有性能数据均来自 Vivado 综合报告与功能级 RTL 仿真结果，确保计算指标具有工程可实现意义。通过与文献实现方案的对比，本作品在保持功能正确性与模块稳定性的前提下，提升了资源利用率，在下文中将对仿真结果进行介绍。

4.2 测试环境

本作品所使用的软硬件测试环境配置如表4.3 所示，包括 RTL 仿真工具、平台主机参数、Python 后端调用环境等，利用 Verilog 语言实现硬件结构描述，并以仿真平台进行测试以评估性能。为全面验证本作品的功能正确性、模块可用性与协同性能，本作品从三个层次开展测试与分析，分别为：关键模块验证、系统功能测试与性能评估。

关键模块测试验证主要包括功能级 RTL 仿真与输出正确性比对，确保关键硬件

模块在输入激励下可稳定运行，且结果与标准软件实现保持一致，确保功能实现。

系统功能测试通过仿真调用完成两个算法的完整流程，验证系统级加解密和签名流程结果的正确性。

性能评估测试则关注各模块与系统整体在协同架构下的效率表现，包括时钟周期、面积资源占用与加速效果等。

表 4.3　软硬件平台配置环境

环境类别	软件/系统名称	配置说明
主机平台环境	系统版本	Windows 11
	软件环境	Python 3.10 + `ctypes`
	编译器	GCC 11.4.0
	软件参考源	pqm4
	测试主机	Intel Core i7-6050P
RTL 测试与仿真平台	仿真工具	Vivado XSIM
	仿真方式	模块级功能仿真
	封装调用模式	动态库

4.3 关键模块测试

4.3.1 keccak 模块测试

首先对 Keccak 模块进行模块级功能仿真测试。以 SHAKE-256 配置为例，测试激励采用 NIST 官方提供的标准向量，输入数据为 `9F2FCC7C 90DE090D 6B87CD7E 9718C1EA 6CB21118 FC2D5DE9 F97E5DB6 AC1E9C10`,期望摘要值为 `2F1A5F71 59E34EA1 9CDDC70E BF9B81F1 A66DB406 15D7EAD3 CC1F1B95 4D82A3AF`。

填充后的消息通过 `Testbench` 发送至硬件系统内存，经分组吸收、置换与压缩等阶段处理后输出摘要结果。图 4.1 给出了基于 VCS 仿真平台获得的波形结果。可以看出，硬件模块在 SHAKE-256 模式下完成了 24 轮迭代计算，最终输出的摘要值与参考值完全一致，验证了硬件实现的功能正确性。对应的官方 C 语言实现结果如图 4.2 所示，二者在输出比特级完全匹配。

在其他配置下，调整 `rate` 与 `output_len` 参数并更换测试向量后，硬件输出同样与软件参考值保持一致，进一步证明了设计在不同参数条件下的通用正确性。

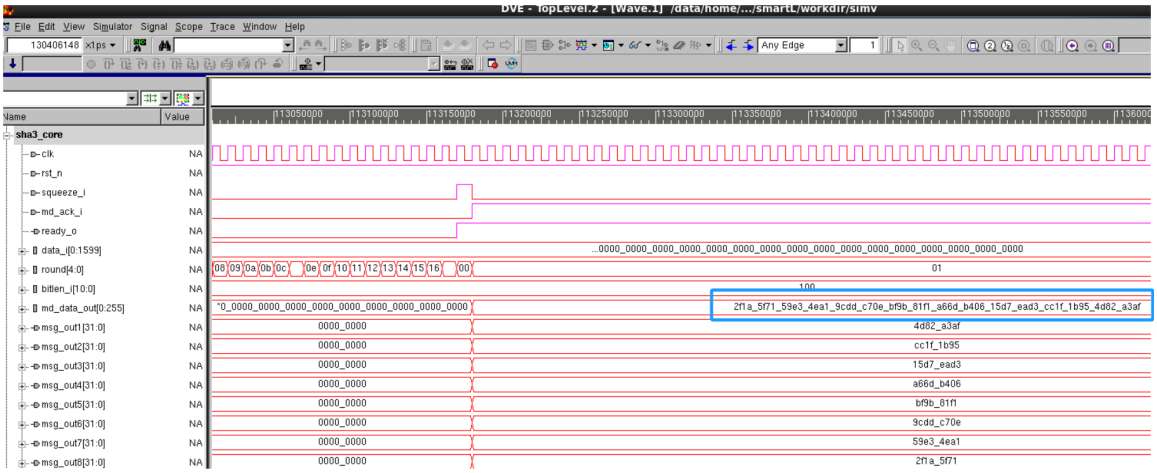


图 4.1　SHAKE-256 硬件模块级仿真结果

```
File  Edit  View  Search  Terminal  Help
qiuyafei>make
gcc -Wall -O3  -c sha3.c -o sha3.o
gcc -Wall -O3  -c main.c -o main.o
gcc  -o sha3test sha3.o main.o
qiuyafei>./sha3test
sha len = 256
message: 9f2fcc7c90de9d6b87cd7e9718c1ea6cb21118fc2d5de9f97e5db6ac1e9c10
sha3-256 value: 2f1a5f7159e34ea19cddc7ebf9b81f1a66db4615d7ead3cc1f1b954d82a3af
```

图 4.2　SHAKE-256 软件实现输出结果

4.3.2 多项式测试

为验证多项式矩阵乘加操作下的功能正确性与调度完整性，我们基于 Xilinx Vivado 仿真平台构建功能级 testbench，对多项式计算模块的 NTT 变换、点乘乘法、累加回写与 INTT 还原全过程进行 RTL 仿真验证。

以 Kyber 参数组为例, 如图 4.3所示, 仿真设定多项式维度为 256、模数为 3329, 验证一次标准的矩阵—向量乘法流程。数据由 `init_polys.hex` 预加载至片上存储，系统按如下阶段执行：

装载阶段：读取矩阵行与向量数据，完成系数准备与参数配置；

计算阶段：执行 NTT→ 点乘与模约简 →INTT 的顺序流水，双算子并行；

回写阶段：将乘积结果写回片上存储并对外输出；

结束判定：完成标志置位，流程终止。

将仿真输出结果图4.4与官方 C 语言实现的结果图4.5进行比对，结果显示无误差。

图 4.3　Kyber 矩阵多项式运算仿真 1

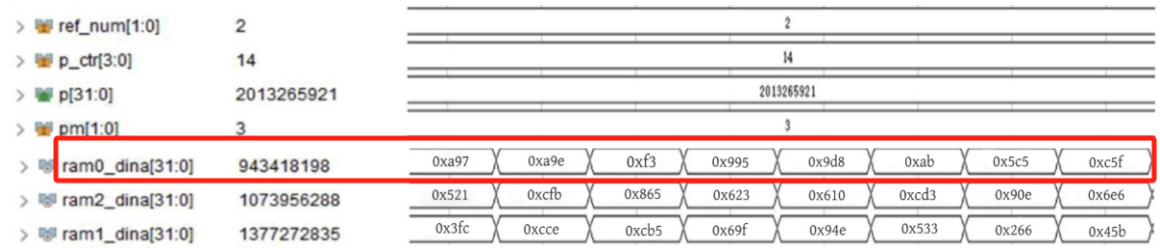


图 4.4　Kyber 矩阵多项式运算仿真 2

```
c = [
  0xa97, 0xa9e, 0xf3, 0x995, 0x9d8, 0xab, 0x5c5, 0xc5f, 0x2fe, 0x9dc, 0x3f5, 0xb85, 0x98e, 0xbf6, 0x350,
0x98c,
  0xc04, 0x3ef, 0xb60, 0x68a, 0x6bf, 0x231, 0x8bc, 0x17f, 0x838, 0xc4c, 0xb00, 0x659, 0xacc, 0x440, 0x205,
0xc72,
  0x6e2, 0xc1f, 0x178, 0x48e, 0x783, 0xa8f, 0x120, 0xb74, 0x7b2, 0xaf, 0x1d4, 0x522, 0x5e1, 0xad3, 0x3c7,
0xb0a,
  0x41a, 0xe1, 0x4cd, 0x321, 0xa8f, 0xc37, 0x767, 0xc21, 0x980, 0xabe, 0x1e2, 0x5f4, 0xb35, 0x6e9, 0x12b,
0x213,
  0x547, 0x818, 0x91b, 0x2fb, 0x301, 0xc6e, 0xc75, 0x140, 0x807, 0x9b3, 0x6d, 0x98d, 0x4e4, 0x903, 0x4d3,
0x71a,
  0x63e, 0x13d, 0x6e9, 0x3e, 0x45e, 0xcf7, 0xb14, 0xb, 0xafb, 0x5e3, 0x5ab, 0x649, 0xb9c, 0xbe0, 0x495,
0x666,
  0xa2c, 0x692, 0x803, 0x9e5, 0x2e5, 0x1bd, 0x67d, 0x31b, 0x5a7, 0x498, 0x111, 0xa1, 0xf8, 0x78d, 0x59d,
0xaed,
  0x431, 0xa8b, 0x2f3, 0x97d, 0x1d5, 0x24e, 0x9d, 0x750, 0xe7, 0x431, 0x805, 0x771, 0x274, 0x796, 0xac2,
0x1c7,
  0x6b3, 0xad, 0x6b2, 0xadc, 0xabb, 0xcf, 0xa10, 0xa4c, 0x858, 0x91, 0x4de, 0x5a7, 0x877, 0xc66, 0x99c,
0xa7,
  0x16a, 0x371, 0x4eb, 0x4de, 0x5ea, 0x4f9, 0x699, 0xf5, 0x66e, 0x79d, 0x7f, 0x65e, 0x8dc, 0x378, 0x567,
0x9e0,
  0x83e, 0x483, 0xcdc, 0x115, 0x26d, 0x1c9, 0xccc, 0x92d, 0x6db, 0xe7, 0x322, 0x83b, 0x915, 0x48a, 0x8e3,
0xb2c,
  0x824, 0x7b, 0x7b2, 0x2b4, 0x838, 0xb8c, 0x608, 0x4d0, 0x420, 0x4c2, 0x59c, 0x8c7, 0x5b8, 0xc7a, 0x818,
```

图 4.5　Kyber 矩阵多项式运算仿真 3

4.4 系统正确性测试

本作品设计了完整的交互界面，以实现后量子密码算法的全流程可视化验证。本节以 Kyber 算法为例展示系统的功能测试过程，并在相同配置下验证 Dilithium 算法，以说明系统在不同算法下的通用性与正确性。

以 Kyber512 参数组为例，系统在“密钥生成—密钥封装—密钥解封装”三个阶段完成完整的交互验证流程。

图 4.6　Kyber 密钥生成结果

①**密钥生成：**点击“生成密钥”按钮后，系统调用硬件仿真 DRAM PUF 模块产生主种子，并派生子采样种子，最终输出公钥与私钥如图 4.6 所示。

②**密钥封装：**上传待封装的 256 比特对称密钥，系统利用公钥进行加密并生成密文文件，封装结果如图 4.7 所示。

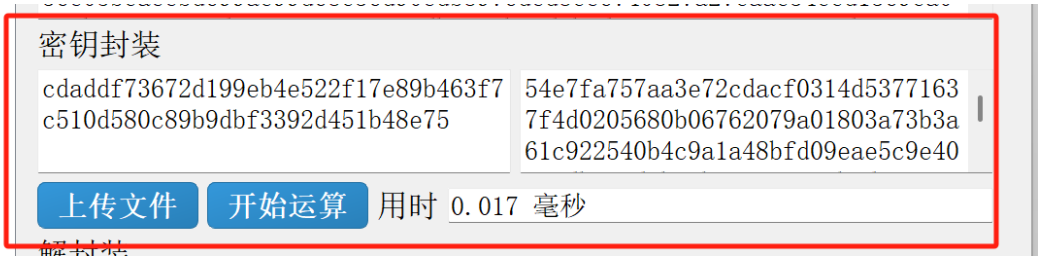


图 4.7　Kyber 密钥封装结果

③**密钥解封装：**将密文文件输入系统后，利用私钥执行解封装。解密结果如图 4.8 所示，与原始明文完全一致，验证了 Kyber512 算法的功能正确性。

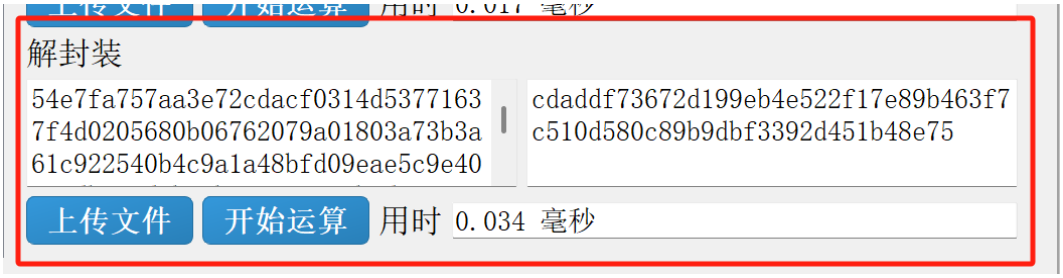


图 4.8　Kyber 密钥解封装结果

对 Kyber768 与 Kyber1024 等其他参数组进行相同测试后，系统均能正确生成密

钥对并完成加解密，证明了算法模块的可配置性与稳定性。

Dilithium 系列算法的测试流程与 Kyber 类似，主要包括密钥生成、消息签名与结果验签三阶段。以 Dil_2 为例，系统可成功输出公私钥对、生成签名并验证有效性如图4.9所示，与官方软件实现结果完全一致。在 Dil_3 与 Dil_5 等不同参数配置下测试结果一致，进一步验证了系统在签名类算法中的正确性与通用性。

图 4.9　Dilithium 签名结果示例

综上，本作品成功实现了 Kyber 与 Dilithium 算法在不同参数配置下的全流程可视化，表明其在密钥生成、加解密及签名验签等环节均具有功能完整性与稳定性。

4.5 总体性能测试

在验证系统各模块和总体功能正确性后，进一步对其运行性能进行评估，重点分析底层核心模块与完整密码算法在不同实现方式下的响应速度与处理效率。通过模块级与算法级两层性能测试，全面展示本作品在软硬件协同架构下的实际运行优势。

4.5.1 核心模块性能测试

为验证核心模块在 FPGA 平台下的综合性能，对 Keccak 哈希模块与多项式运算模块进行了资源消耗与频率评估。所有数据均基于 RTL 综合结果与工具报告提取，主要指标包括逻辑资源（LUT、FF、BRAM）、最大工作频率（Fmax）、时延及吞吐率。与现有文献结果的对比如表 4.4 与表 4.5 所示。

从结果可见，本作品在有限逻辑资源条件下实现了更高的运算效率：

- Keccak 模块在 225 MHz 下可实现 6.71 Gbps 吞吐率，资源使用较文献 [20] 减少约 21%LUT 与 42%FF，体现出较优的面积—性能平衡；
- 多项式运算模块采用 D-KPolyArch 架构，通过双 ALU 解耦与四通道交织存储实现 208 MHz 的运行频率，256 维运算时延仅 22.86 μs，较文献 [21] 减少两个数量级；
- 整体而言，两模块在保持高频与正确性的同时显著降低资源占用，适合低功耗 FPGA 平台部署。

表 4.4　各 Keccak 实现方案资源与性能对比

实现	平台/工艺	LUT(个)	FF(个)	Fmax(MHz)	吞吐率 (Gbps)	面积效率
本作品	Vivado 仿真	2,500	2,100	225	6.71	11.6
文献 [20]	Virtex-6	3,166	3,642	359	11.62	13.71
文献 [22]	SMIC 65nm	–	–	–	0.55	–
文献 [23]	Virtex-5	2,405	1,882	202	4.14	10.6
文献 [24]	Virtex-6	2,849	2,212	265	5.43	12.6
文献 [25]	Cyclone-IV	2,889	2,317	160	1.85	–

表 4.5　各多项式实现方案资源与性能对比

实现	平台/工艺	LUT(个)	FF(个)	BRAM(块)	Fmax(MHz)	时延 (μ s)
本作品	Vivado 仿真	1926	1312	12	208	22.86
文献 [26]	Artix-7	2382	1381	10	150	–
文献 [21]	Artix-7	432	278	4	125	1007.79
文献 [27]	Zynq-7000	886	618	1	–	239.72
文献 [28]	Zynq-7000	2942	4881	0	153	28.44

4.5.2 系统性能测试

在验证模块功能后，对整体系统进行算法级性能评估。测试对象涵盖 CPU、GPU 及本作品平台，比较其在 Kyber 与 Dilithium 算法下的典型操作吞吐率（ops/s）。对比平台包括 PQMagic（优化型 CPU 实现）、liboqs、botan、boringssl（通用密码库实现）及 cuPQMagic（GPU 实现），结果如表4.6 与表4.10 所示。

表 4.6　不同密码库每秒操作次数对比（单位：ops/s）

每秒操作次数	PQMagic	liboqs	botan	boringssl	本作品	cuPQMagic
签名密钥生成	52501	23422	5768	6252	163732	233078
数字签名	23697	12557	3537	1411	93298	135730
签名验证	49144	24067	25137	5522	179621	249373
封装密钥生成	159207	92625	20088	28108	309861	1133114
密钥封装	154704	89307	72433	34132	756106	1068131
解封装	137517	90962	43582	34292	433511	653727

从对比结果可见：

- 相比通用 CPU 实现（liboqs、botan、boringssl），本作品平均性能提升 3–7 倍；
- 相比优化型 CPU（PQMagic），性能提升约 3–4 倍，在签名验证与密钥封装任务中尤为显著；
- 与 GPU 实现（cuPQMagic）相比，本作品平均达到其 65%–75% 的性能水平，兼具高吞吐与低面积优势。

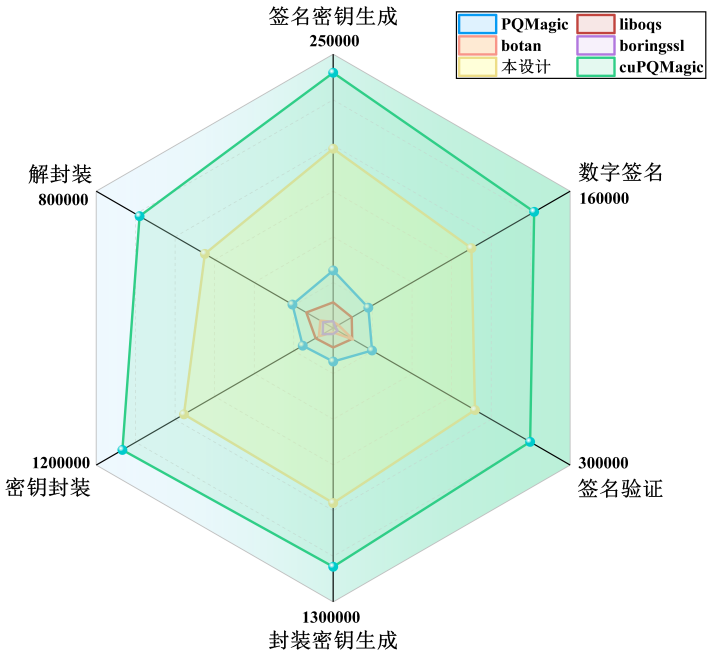


图 4.10　系统性能对比

图4.10展示了本作品与其他实现方案的性能对比。可以看出，本作品在理论性能上已接近 GPU 实现，显著优于各类 CPU 实现，说明其硬件架构已达到工程级性能水准。此外，表4.7对比了本作品与现有车联网级密码芯片（如国芯 CCM3310S 系列）的性能指标：本作品在资源开销、运算效率和安全等级等方面均满足车载平台约束：表明系统具有良好的车联网级落地潜力。

表 4.7　与 CCM3310S-T 车载芯片性能对比

评价维度	本作品指标	CCM3310S-T 芯片性能参考
资源占用	不超过 5,000 个 256-LUT	小于芯片预设 16KB ROM
运算效率	各组件时延约 10^{-5}s 量级	全流程与芯片一致，均为 ms 量级
安全性	对应 RSA-3072 等级安全强度	强于芯片最高安全配置 RSA-2048

第五章 总结与展望

本作品面向智能网联汽车的量子安全信息安全需求，首次提出适配嵌入式部署的后量子密码算法融合优化方案，以总面积最小化为约束、计算效率最大化为目标构建规划求解模型，实现系统整体计算效率的全局最优，并基于 DRAM-PUF 实现多级随机数种子管理，降低随机数种子泄露风险，核心创新主要体现在以下几个方面：

（1）Keccak 哈希加速器深度重构，突破吞吐瓶颈与路径延迟限制

提出双阶段流水的层次调度架构，解决传统实现中逻辑深度长的痛点；同时，基于硬件特性，首次实现零逻辑开销的重排驱动执行路径。在保证多模式兼容的前提下，整体吞吐提升至 6.71Gbps，LUT 占用减少 21%，触发器数量下降 42%，在吞吐与资源效率之间取得良好平衡，显著增强系统级可扩展性与模块可复用性。

（2）多类型采样模块统一调度，采样效率与控制面积双优化

为四类标准类型构建统一采样路径结构并支持动态配置切换，仅用 1 个 DRAM 完成全类型采样输入，种子存储压缩超 80%。控制路径采用三级寄存器结构实现缓存、移位与输出复用，减少控制冲突与资源浪费。测试表明控制信号减少 66%，寄存器节省 17 个，实现高效可重构的采样模块设计。

（3）存算一体多项式架构设计，兼容双算法并行与存储压缩需求

构建 D-KPolyArch 架构，在存储与运算层协同优化，解决多项式运算中访存冲突与资源重复问题。采用交织式 4-Bank 存储与混合进制地址映射，支持 NTT 无冲突并发访存，多项式矩阵存储压缩率达 87.5%；运算层构建双 ALU 解耦结构适配不同乘法需求，实现对多类运算的统一支持，吞吐提升 1.9 倍，降低资源占用与调度复杂度。

在后续研究中，本作品将从以下几个方面进一步拓展与完善：1）**硬件安全防护：**针对核心算子深入开展侧信道攻击风险分析，结合现有控制框架引入时序随机化与数据扰动等对抗机制，提升硬件安全性。2）**国密协议适配：**在保持后量子算法模块化设计的基础上，进一步推动核心算子与国密协议及国密算法的对接，实现统一接口支持，为后续国密—后量子混合密码协议提供硬件支撑。3）**规划求解模型扩展：**基于现有的算子级规划求解框架，研究其在更多规模嵌入式系统中的可扩展性，将资源优化机制推广至其他可信计算模块。

综上所述，本作品针对车规级嵌入式平台的量子信息安全需求，提出了双算法协同加速方案，填补了格算法在嵌入式应用中的部署空白，具有良好的跨平台适配性。随着欧盟 PQC 转型路径图的发布，本作品为车载平台实现后量子密码过渡提供了技术保障，推动量子安全技术在智能网联汽车领域的应用。

参考文献

[1] 冯凯, 李巍, 龚洁中. 车联网中密码算法应用现状分析. 中国信息安全, (09):97–99, 2019.

[2] Richard Soja. Automotive security: From standards to implementatio, 2016.

[3] 丘敬云, 吴祥晨, 周宇坤, and 顾泽南. 国密 sm2、sm4 混合算法在车联网中的应用研究. 仪器仪表用户, 30(8):9–12,99, 8 2023.

[4] 李建立, 莫燕南, 粟涛, and 等. 基于国密算法 sm2、sm3、sm4 的高速混合加密系统硬件设计. 计算机应用研究, 39(9):2818–2825,2831, 2022. Li Jianli, Mo Yannan, Su Tao, et al. Hardware design of high-speed hybrid encryption system based on SM2, SM3 and SM4 algorithm.

[5] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - a new hope. Cryptology ePrint Archive, Paper 2015/1092, 2015.

[6] Tobias Stelzer, Felix Oberhansl, Jonas Schupp, and Patrick Karl. Enabling lattice-based post-quantum cryptography on the opentitan platform. In *Proceedings of the 2023 Workshop on Attacks and Solutions in Hardware Security*, ASHES '23, page 51–60, New York, NY, USA, 2023. Association for Computing Machinery.

[7] Shiyang He, Hui Li, Fenghua Li, and Ruhui Ma. A lightweight hardware implementation of crystals-kyber. *Journal of Information and Intelligence*, 2(2):167–176, 2024.

[8] 敖思凡, 王后珍, 白鹭, 文嘉明, and 张焕国. Crystals-dilithium 算法实现的空间优化. 武汉大学学报 *(*理学版*)*, 69(06):709–718, 2023.

[9] 孙俊彦. 后量子密码算法 crystals-kyber 的硬件设计研究. Master's thesis, 中国科学技术大学, 2024.

[10] Denisa O. C. Greconici, Matthias J. Kannwischer, and Amber Sprenkels. Compact dilithium implementations on cortex-m3 and cortex-m4. Cryptology ePrint Archive, Paper 2020/1278, 2020.

[11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Keccak. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 313–314. Springer, 2013.

[12] Miroslav Knezevic, Frederik Vercauteren, and Ingrid Verbauwhede. Faster interleaved modular multiplication based on barrett and montgomery reduction methods. *IEEE Transactions on Computers*, 59(12):1715–1721, 2010.

[13] Ramesh C Agarwal and C Sidney Burrus. Number theoretic transforms to implement fast digital convolution. *Proceedings of the IEEE*, 63(4):550–560, 2005.

[14] Xiang Feng and Shuguo Li. Accelerating an fhe integer multiplier using negative wrapped convolution and ping-pong fft. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 66(1):121–125, 2018.

[15] Soubhagya Sutar, Arnab Raha, and Vijay Raghunathan. D-puf: An intrinsically reconfigurable dram puf for device authentication in embedded systems. In *Proceedings of the International Conference on Compilers, Architectures and Synthesis for Embedded Systems*, pages 1–10, 2016.

[16] Lenos Ioannou, Harris E. Michail, and Artemios G. Voyiatzis. High performance pipelined fpga implementation of the sha-3 hash algorithm. In *2015 4th Mediterranean Conference on Embedded Computing (MECO)*, pages 68–71, 2015.

[17] 张聪. 基于格的后量子密码高效硬件实现研究. PhD thesis, 华中科技大学, 2021.

[18] Neng Zhang, Bohan Yang, Chen Chen, Shouyi Yin, Shaojun Wei, and Leibo Liu. Highly efficient architecture of newhope-nist on fpga using low-complexity ntt/intt. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020, Issue 2:49–72, 2020.

[19] Stefano Di Matteo, Ivan Sarno, and Sergio Saponara. Cryphtor: A memory-unified ntt-based hardware accelerator for post-quantum crystals algorithms. *IEEE Access*, 12:25501–25511, 2024.

[20] 刘冬生, 陈勇, 熊思琦, 杨朔, and 胡昂. 应用于后量子密码的高速高效 sha-3 硬件单元设计. 信息安全学报, 6(6):32–39, 12 2021.

[21] Yufei Xing and Shuguo Li. An efficient implementation of the newhope key exchange on fpgas. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 67(3):866–878, 2020.

[22] 张跃军, 廖澴桓, and 丁代鲁. 基于 lut 的高速低硬件开销 sha-3 算法设计. 电子技术应用, 43(4):43–46, 1 2017.

[23] 田鑫. 基于 fpga 的 sha-3 算法硬件实现优化与系统设计. 硕士论文, 西安电子科技大学, 2019.

[24] 黄炳权. Risc-v 平台下高效可配置的哈希算法硬件实现. 硕士论文, 广东工业大学, 2024.

[25] 王振江. 面向 risc-v 处理器的 sha-3 算法协处理器设计. 硕士论文, 中原工学院, 2024.

[26] Po-Chun Kuo, Wen-Ding Li, Yu-Wei Chen, Yuan-Che Hsu, Bo-Yuan Peng, Chen-Mou Cheng, and Bo-Yin Yang. High performance post-quantum key exchange on FPGAs. Cryptology ePrint Archive, Paper 2017/690, 2017.

[27] Tim Fritzmann, Uzair Sharif, Daniel Müller-Gritschneder, Cezar Reinbrecht, Ulf Schlichtmann, and Johanna Sepulveda. Towards reliable and secure post-quantum co-processors based on risc-v. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1148–1153, 2019.

[28] Tobias Oder and Tim Güneysu. Implementing the newhope-simple key exchange on low-cost fpgas. In Tanja Lange and Orr Dunkelman, editors, *Progress in Cryptology – LATINCRYPT 2017*, pages 128–142, Cham, 2019. Springer International Publishing.